Design, Simulation, and Implementation of an Affordable Farm Maintenance Robot for Small- and Medium-Scale Farmers

Meenal Shah

A Project Report
submitted in partial fulfillment of the
requirements for the degree of

Master of Science

University of Washington

2026

Committee:

Min Chen, Committee Chair

Clark Olson, Committee Member

Michele B. Price, Committee Member

Program Authorized to Offer Degree:
Computer Science and Software Engineering

University of Washington

Abstract

Design, Simulation, and Implementation of an Affordable Farm Maintenance Robot for Small- and Medium-Scale Farmers

Meenal Shah

Chair of the Supervisory Committee:
Min Chen
Department of Computing & Software Systems

Small and medium farms need timely field data—soil moisture, temperature, humidity, and early plant disease symptoms. Although autonomous agricultural robots exist, they are designed for large farms and typically cost thousands of dollars. This project presents the design, simulation, and implementation of a compact 25 cm $\times$ 25 cm autonomous ground robot for crop monitoring and leaf disease detection, built for under $1,000. The robot follows a boustrophedon coverage pattern, samples environmental and soil parameters at fixed intervals, captures leaf images for on-device disease inference, avoids obstacles using IR sensing, and returns to a docking station to charge and upload data to a cloud-backed dashboard. Control is implemented as a finite state machine for predictability and low compute cost. Alongside the prototype, this work evaluates whether bio-inspired AI can support lightweight on-robot disease detection: three nature-inspired feature selectors and five classifiers spanning traditional CNNs and bio-inspired architectures were trained on a PlantVillage subset covering twelve diseases and three crops. LiteCShuffle achieved the highest accuracy (97.63%) at a model size under 2 MB, beating both traditional and bio-inspired models. This suggests that the bio-inspired models evaluated, which were originally designed for non-agricultural perceptual tasks, do not transfer directly to leaf disease classification without further adaptation. The prototype demonstrates that meaningful autonomous crop monitoring is achievable at a price point compatible with small-farm budgets.

TABLE OF CONTENTS

Page

LIST OF FIGURES

LIST OF TABLES

ACKNOWLEDGMENTS

I would like to express my deepest gratitude to my committee chair, Dr. Min Chen, for her invaluable guidance, support, and encouragement throughout this project. Her insight and mentorship were instrumental in shaping the conception of this work, guiding its development, and bringing it to completion. I am sincerely thankful for her patience, expertise, and continued belief in my abilities.

I am also grateful to my committee members, Dr. Clark Olson and Dr. Michele B. Price, for their time, thoughtful feedback, and willingness to serve on my committee. Their perspectives and suggestions significantly strengthened this work.

I would like to extend my appreciation to the Makerspace at the University of Washington Bothell for providing access to resources, tools, and a collaborative environment that made this project possible. The knowledge, hands-on experience, and assistance I received there gave me the confidence and capability to undertake and complete this work.

I am thankful to SeattleMakers for their early guidance and encouragement, which helped me take the first steps toward building my prototype and set a strong foundation for this project.

I also wish to acknowledge the faculty, administration, and staff at the University of Washington Bothell. The opportunities, resources, and guidance they provided have played an important role in shaping my academic journey and personal growth.

Lastly, I am deeply grateful to my family and friends for their unwavering support, motivation, and encouragement, which enabled me to persevere and give my best throughout this journey.

I sincerely appreciate everyone who contributed to making this work possible.

Chapter 1
INTRODUCTION

Agriculture is increasingly dependent on timely field information: soil moisture, plant health, light conditions, temperature, humidity, and early disease symptoms. For small and medium farms, the challenge is not the absence of technology but its accessibility: the most capable systems are designed for large-scale operations, require specialized infrastructure, or impose cost and maintenance burdens that make adoption difficult. Agricultural workers also face health risks from heat exposure, repetitive motion, chemical contact, and long working hours, making automation valuable not only as a productivity tool but also as a worker-safety mechanism[1].

Several autonomous agricultural platforms already exist, including commercial precision-farming robots and research-grade crop monitoring systems. However, because most existing systems emphasize large acreage, commercial-grade machinery, or expensive sensing stacks, there is a clear gap for a small, inexpensive robot that can gather useful local measurements, navigate crop rows, detect plant disease symptoms, and return to charge without requiring a large capital investment. The goal of this work is therefore to create a compact agricultural robot prototype and to study whether biological systems can further support lower-resource autonomy. Several biological systems—particularly insects—navigate cluttered environments, make rapid decisions with limited neural resources, use visual cues efficiently, and recover from environmental uncertainty[2]. These traits map directly to agricultural robotics, where low-cost robots must move across uneven terrain, avoid obstacles, conserve energy, and make useful decisions despite limited computing power.

1.1 Significance

The significance of this project is strongest for farmers who cannot justify large autonomous machinery but still need better field visibility. Small and medium farmers need tools that

reduce manual scouting time, detect early crop stress, and provide digital records without adding complicated workflows. A small robot does not replace full-scale farm machinery; instead, it acts as a low-cost field assistant that can gather data repeatedly and consistently.

A ground robot was chosen over an aerial platform for several reasons. For crop monitoring, drones are limited to measuring the plant canopy, since that is all they can observe from above. While canopy data is sufficient for some applications, significant information can be gathered only by examining fruit and sampling the soil directly. Drones are also hindered by weather conditions such as strong winds, rain, or fog, and their run times are limited by battery capacity and high power consumption. Regulatory factors further constrain drone use: under current Federal Aviation Administration (FAA) policy, drones must be operated by a licensed human pilot, which limits their ability to function as fully autonomous field assistants and adds time and labor cost for the farmer. Finally, the initial investment in drones, along with ongoing maintenance and operator-training costs, can be a substantial financial burden for small and medium farmers[3].

1.2 Bio-Inspired AI

A secondary goal of this research is to evaluate whether existing bio-inspired technology, particularly in AI, can contribute to building an affordable autonomous farm-maintenance robot.

Bio-inspired robotics is relevant because animals and insects solve navigation and perception problems with extremely limited energy and computational resources. Insect-inspired AI research argues that small autonomous robots can benefit from efficient biological principles rather than only scaling down large robotic architectures[2]. Swarm and insect-inspired algorithms such as artificial bee colony, grasshopper optimization, and related hybrid optimizers have already been applied to plant disease detection and feature optimization[4], [5]. This work investigates whether such biological ideas meaningfully reduce cost, complexity, and resource use in current practice.

1.3 Goals & Specifications

The goal is to design and evaluate a compact ground robot that can cover a row-planted field, collect environmental and plant-health data, detect visible leaf disease symptoms, return to a charging station, and upload results to a cloud-backed application. The specifications and evaluation criteria, described in Table 1.1, are intentionally tied to testable behaviors so the prototype can be evaluated in simulation and hardware trials.

Metrics	**Purpose**
Autonomy Level	Ability to self-navigate without manual intervention
Sensor Accuracy	Precision in identifying soil and plant health conditions
Battery Life	The number of hours the robot operates on a full charge
Charging Efficiency	Time required for a full charge
Data Communication Speed & Reliability	Time and error rate in transferring data
Maintenance Simplicity	Time and ease of diagnosing and repairing hardware issues

Table 1.1: *Metrics to calculate the effectiveness of the agriculture-robot*

1.4 Paper Structure

The remainder of this paper is organized as follows. Chapter 2 presents the literature survey and design research, evaluating existing agricultural robotic systems, reviewing biologically inspired AI models suitable for computer vision and disease detection, and comparing path-coverage patterns and navigation strategies relevant to row-crop fields. It also discusses hardware design choices for traversing tilled terrain. Chapter 3 describes the selected hardware and software design of the robot, along with the experimental setup for the AI model comparison. Chapter 4 presents the experimental results and discussion. Chapter 5 concludes the work and outlines directions for future improvement.

Chapter 2

LITERATURE SURVEY AND DESIGN RESEARCH

2.1 Existing Systems

A number of farm-maintenance robots already exist in both research and commercial settings, performing functions such as pesticide spraying, weeding, and phenotyping. This section provides a broad overview of some of the systems involved in this space in terms of functionality, size, and estimated costs, summarized in Table 2.1.[3], [6], [7], [8], [9]

Atefi et al. review a range of these systems, including large four-wheeled robots like BoniRob and ground platforms like Ladybird and Thorvald II that carry multispectral, hyperspectral, thermal, and LiDAR sensors to measure traits such as plant height, biomass, canopy closure, and NDVI. Smaller platforms include Vinobot and Robotanist, which use robotic arms for measuring plant dimensions and stalk strength, and the low-cost TerraSentia, designed to navigate under the crop canopy. The review notes a general trend toward modular, multi-sensor designs while highlighting ongoing challenges in navigation, real-time data processing, and cost reduction.[6]

Two other research systems focus on integrated monitoring and field management. Linford and Haghshenas-Jaryani's ground mobile manipulator combines a 4WD Rover Pro base with a 6-DOF arm whose end-effector houses a soil probe, RGB-depth camera, and photoresistor–LED insertion-feedback assembly; it fuses RTK GPS, LiDAR, and IMU data to achieve consistent soil measurements with a 78-second average cycle per location[3]. Cui et al.'s modular field management robot emphasizes adaptability, offering four switchable steering modes, an adjustable wheel-track for varying row spacing, and interchangeable attachments for spraying, weeding, or sensing, with navigation handled by vision-based crop-row following and GPS-assisted path planning.[7]

Finally, two systems target disease detection and commercial deployment. RobHortic is a remote-controlled ground robot validated over three growing seasons for detecting infection

in carrot fields, using an 8-band multispectral camera, RGB camera, and standardized illumination to achieve detection rates of 66.4% in the laboratory and 59.8% in the field; its chassis costs under €5,000, though the full system is considerably more expensive[8]. The commercially available FarmDroid FD20 is a fully autonomous, solar-powered robot weighing 1,250 kg that performs simultaneous seeding and mechanical weed control across more than 100 crop types, recording each seed's coordinates to enable precise intra-row weeding and optional micro-spraying that can reduce agrochemical use by up to 94%.[9]

Table 2.1 clearly shows that even the lowest-cost platform requires substantial capital investment. Many systems in this space are additionally moving toward multi-robot coverage, further increasing costs. These robots are typically designed for large-scale farms and are not practical for small or medium-scale farmers. Building on this existing research, this work aims to close the gap by emphasizing affordability, compact size, and biological inspiration.

Parameter	NMSU Mobile Manipulator	Multifunctional Field Management Robot	FarmDroid FD20	RobHortic	Representative Phenotyping Platforms
Robot type	Mobile manipulator (UGV + 6-DOF arm)	Reconfigurable electric UGV	Commercial autonomous seeding/weeding platform	Remote-controlled proximal sensing UGV	Various UGVs (wheeled/tracked)
Dimensions (rover/base)	62 × 39 × 25.4 cm (rover base)	1 × 1.8 × 2 m[1]	3.5 m working width; handles row spacings of 22.5–90 cm; up to 12 active rows	Four fat-bike wheel chassis; compact enough for row-crop navigation	Varies widely; BoniRob ∼1.8 m × 1.2 m; TerraSentia ∼33 cm wide
Total system mass	∼21–25 kg (11 kg rover + 10 kg arm + payload/electronics)	Heavy (based on size and sensor weights)	1250 kg	∼50–100 kg estimated for chassis + sensors	70 kg (open-source phenotyping rover); >500 kg (larger gantry-type systems)
Max speed	2.5 m/s	Not reported	0.26 m/s (950 m/h)	1 m/s	Varies: 0.5–1.5 m/s
Power source	LiPo battery (TATTU 12,000 mAh, 22.2 V, 6S)	Electric (battery-powered)	Solar panels (4×, max 1.6 kWh output); 18–24 hr continuous operation	Electric (battery-powered)	Electric/battery; some solar-assisted
Navigation / localization	RTK GPS (simpleRTK2B, <1 cm accuracy); LiDAR SLAM (RPLIDAR S1); IMU (WT901BLECL); EKF sensor fusion; ROS2 (Humble)	Vision-based crop-row following; GPS-assisted; four steering modes (Ackermann, four-wheel, crab, zero-radius)	RTK GPS (8 mm precision); fully autonomous path following based on seed coordinates	GNSS geolocation; remote-controlled	GPS/GNSS; visual SLAM; RTK GPS in high-precision systems
Primary sensors	TEROS 12 (soil moisture/temperature/EC, ±0.03 m^3/m^3 VWC); Intel RealSense D435i RGB+D camera (×2, <2% at 2 m); RPLIDAR S1 (40 m range, ±5 cm); IMU; RTK GPS; photoresistor + LED (soil insertion feedback); ultrasonic sonar (HC-SR04, ±3 mm)	Camera system for vision-based navigation; modular sensor payload (spraying booms, weeding tools, crop information collectors)	RTK GPS receiver (8 mm); seed detection encoder; mobile app interface	Multispectral camera (CMS-V, Silios Technologies, 8 spectral bands); color RGB camera; hyperspectral imager (400–1000 nm); GNSS; encoder	Hyperspectral, multispectral, RGB-D, thermal cameras; LiDAR; stereo cameras; force sensors (platform-dependent)
Software / OS	ROS2 (Humble); Python/C++ nodes; custom soil insertion and plant detection algorithms	Custom navigation controller	Proprietary FarmDroid app	Custom image acquisition and synchronization software	ROS (various); custom phenotyping software
Primary functions	In-situ soil moisture/temperature/EC measurement; crop image capture (15 poses); plant height/base detection (HSV colour filtering); autonomous slope-compensated soil probe insertion	Field spraying; mechanical weeding; crop information collection; multiple steering-mode path planning	High-precision autonomous seeding; inter- and intra-row mechanical weeding; targeted micro-spraying (reduces chemical use up to 94%)	Detection of pests and diseases (proximal sensing); field NDVI and multispectral mapping; geolocation of plant imagery	Plant height, LAI, stem thickness, biomass, NDVI, canopy closure, spectral reflectance, stalk strength
Reported accuracy / performance	Soil moisture deviation <3%; plant height estimation error 1.9–2.5%; sampling time 73–97 s per point (avg. 78 s) over 15 field locations	Vision navigation lateral error not explicitly reported; successful trajectory tracking in Ackermann and crab steering modes demonstrated	8 mm GPS seeding precision; up to 6 ha/day coverage; compatible with 100+ crop types	66.4% pest detection rate (laboratory); 59.8% (field); ∼1 image per 80 cm at 1 m/s	System-dependent; generally ±1–5% for geometric traits; varies widely
Estimated or reported cost[2]	∼$12,900–$18,000 USD	∼$8,000–$15,000 USD	€75,000–€95,000	<€5,000 for robot frame and chassis; total system including lab-fabricated sensors estimated at €20,000–€30,000	$5,000–$200,000+ USD depending on platform (TerraSentia ∼$5,000; BoniRob >$100,000)

Table 2.1: *Summary of the cost, specifications and use-case of different farm-maintenance robots*

2.2 Plant Disease AI

De Croon et al. argue that the fundamental constraint for autonomous field robots, i.e., operating for extended periods with onboard compute requires a paradigm shift toward insect-inspired AI. The key principle is parsimony: achieving robust, adaptive behavior with minimal computational resources. Three mechanisms underlie insect intelligence that can be translated to robot AI: embodiment, sensory-motor coordination, and swarming[2].

Embodiment requires co-designing the sensor/actuator apparatus with the control algorithm to offload cognitive computation onto physical dynamics. For plant disease robots, this translates to designing camera rigs and lighting that simplify image preprocessing. Sensory-motor coordination complements this through active sensing—for example, moving the camera to standardize illumination angle—mirroring how insects use neck muscles for gaze stabilization. Finally, swarming distributes monitoring tasks across a fleet of micro-robots, each carrying a minimal model, to collectively achieve high field coverage. The result is a system that is faster, lighter on each individual robot, and more fault-tolerant[2].

Motivated by these principles, this work compares several bio-inspired AI models against traditional computer-vision architectures to identify the most suitable approach for an affordable, lightweight farm-maintenance robot.

2.2.1 Bio-inspired AIs

Spiking Neural Networks and Neuromorphic Computing[10]

Yang et al. (2024) present a fault-tolerant neuromorphic framework integrating visual perception with decision-making via Spiking Neural Networks (SNNs). There are three core contributions relevant to lightweight disease detection: Quadruplet STDP (Q-STDP), Link-Fault Neuromorphic Tolerant (LFNT) routing, and hardware efficiency.

Q-STDP uses a biologically plausible learning rule that updates synaptic weights based on the precise timing of four spikes (quadruplet), using six auxiliary variables (r1, o1, r2, o2,

[1]Estimate based on picture and design description.

[2]For the calculation of estimated cost of different robots, refer to Appendix B.

r3, o3) with exponential decay dynamics. Q-STDP outperforms other STDPs in classification accuracy, especially under noise, because it captures higher-order spike correlations and encompasses more learnable variables for weight plasticity. LFNT is a fault-tolerant routing algorithm for the neuromorphic network-on-chip (NoC) that reduces bypass routing hops, minimizing recovery time when hardware faults occur. LFNT maintains saturation accuracy from 0% to 30% fault rates and achieves substantially higher accuracy than prior works. This model is implemented on an Intel Stratix III FPGA, which uses time-multiplexed physical neurons (replacing multipliers with multiplexers), reducing on-chip memory requirements.

The key advantage of SNNs for edge deployment is event-driven computation: the network fires only when input changes (sparse processing), consuming energy proportional to scene activity rather than at fixed clock rates.

Hybrid WOA_APSO for CNN Optimization[11]

Vijayan & Chowdhary (2025) propose a Hybrid Whale Optimization Algorithm with Adaptive Particle Swarm Optimization (WOA_APSO) for rice crop disease detection. The WOA explores a large, diverse search space through random search agent selection (avoiding premature convergence to local optima), while APSO adaptively adjusts particle velocities for rapid convergence. Together, they select the optimal feature subset for feeding into a CNN classifier from 143000+ features for each image. These features included color moments across 5 different channels, Gray Level Co-occurrence Matrix (GLCM) descriptors, Gray-Level Run Length Matrix (GLRLM) descriptors, Gray Level Dependence Matrix (GLDM) features, Local Binary Pattern (LBP) features, Histogram of Oriented Gradients (HOG) features, and other features encompassing the shape, pixel distribution and texture features of the leaf image.

This approach is particularly valuable for deployments where training data is limited and hand-crafted textural features provide complementary information to deep features.

KCNet: An Insect-Inspired Single-Hidden-Layer Neural Network[12]

Hong and Pavlic (2025) draw directly on the neural architecture of the Drosophila melanogaster mushroom body to propose KCNet, a lightweight neural network that replaces learned dense weights in the input-to-hidden transformation with sparse, randomized, binary weights.

This architecture is fundamentally parsimonious: it requires no iterative weight updates for the dominant input-to-hidden transformation, dramatically reducing training computation. The random expansion (from ∼50 input dimensions to ∼2,000 KC dimensions) is analogous to locality-sensitive hashing, creating a higher-dimensional space where categories become more linearly separable.

2.2.2 Traditional AIs

Tariq et al. (2024) apply VGG16 to classify corn leaf images into four categories—healthy, common rust, blight, and gray leaf spot—combining the model with Layer-wise Relevance Propagation (LRP), an Explainable AI (XAI) technique, to make the model's decision-making transparent. LRP uses backpropagation to assign relevance scores to individual input pixels, highlighting which regions of the leaf image most influenced the classification decision. This transparency is crucial for agricultural deployment, where farmers need to understand and trust model predictions to act on them.

The LiteCShuffle (LCS) paper (2025) addresses a practical gap: while deep learning methods achieve high accuracy on plant disease classification benchmarks, many existing models are too large for deployment on resource-constrained platforms such as IoT or mobile devices. LCS is built on ShuffleNetV2, which achieves efficient feature reuse through channel splitting and channel shuffling. Rather than costly group convolutions, ShuffleNetV2 splits feature channels into two branches at each block—one passes through unchanged, the other is processed—and the branches are then concatenated and shuffled to allow cross-channel information exchange. This strategy achieves strong representational capacity while minimizing Floating Point Operations Per Second (FLOPs) and memory requirements. The paper frames lightweight model design as a necessary step between academic accuracy benchmarks and real-world agricultural deployment on edge hardware[13].

2.2.3 Training and Model Enhancement Strategies

Beyond model architecture itself, several training-stage enhancement strategies emerged during the literature review that are relevant to building accurate, lightweight models. Two are discussed below.

Bag of Tricks[14]

Bansal, Jain, and Khandelwal (2022, Stanford CS231n) evaluate and combine several techniques for faster and more stable image classification training, demonstrating combined speedups of 1.5–2.5× with higher validation accuracy on VGG-16 and ResNet-18. These techniques are architecture-agnostic and applicable to plant disease detection models. Two are most relevant to this work:

- **Importance Sampling:** Standard uniform mini-batch sampling treats all training examples as equally informative. Importance sampling assigns a higher sampling probability to examples that yield more informative gradient updates. The paper evaluates and extends several strategies: bandit-based sampling (AdamBS), class-weighted importance sampling (novel), and momentum-based loss weighting (novel). For plant disease datasets with inherent class imbalance—common diseases vastly outnumber rare ones—importance sampling can meaningfully improve convergence speed and final model accuracy.
- **GradInit (Gradient-Based Weight Initialization):** When a deep or unusual network starts with poorly chosen initial weights, training can become unstable early on—gradients may blow up or shrink to nothing, and the model learns slowly. GradInit fixes this by automatically tuning the starting weights before training begins. It does this in a few steps: it takes a batch of data, performs a single trial training step to see how the weights would update, and then adjusts the scale of the initial weights so that this step reduces the loss on a second, held-out batch. Two safeguards keep the process well-behaved: gradient clipping (which stops gradients from exploding) and a minimum limit on how small the weights can be scaled (which prevents the weights

from collapsing to useless values). The method also uses a partial overlap between the two batches—sharing about half their data—which balances two extremes: using the exact same batch (which biases the result) versus using completely separate batches (which adds noise). GradInit is especially helpful when building custom lightweight networks that mix components from different sources, where standard initialization methods like Xavier or Kaiming may not work well.

Automated Channel Pruning via Artificial Bee Colony[5]

Liu and Zhao (2023) observe that traditional channel pruning methods — which set a fixed pruning rate or sparsity factor before training and apply it uniformly — introduce human judgment into the compression process and frequently produce suboptimal network structures. They propose DPruner, an automated channel pruning algorithm driven by the Artificial Bee Colony (ABC) algorithm, which searches for the globally optimal pruning structure without relying on manual hyperparameter tuning.

Results on CIFAR-10 show that DPruner achieves comparable or better accuracy than the ABCPruner baseline across VGGNet-16, GoogLeNet, ResNet-56, and ResNet-110, with channel count reductions of 0.3–5.7% more than the baseline. FLOPs reduction rates reach 63.4% (VGGNet-16), 67.7% (GoogLeNet), 58.9% (ResNet-56), and 66.9% (ResNet-110). On CIFAR-100, with only small accuracy drops (0.64–1.72% top-1), DPruner achieves channel reductions of 23–37%, FLOPs reductions of 54–66%, and parameter reductions of 51–63% across the same networks. This automated approach is directly applicable to compressing large plant disease detection models for edge deployment.

2.3 Robot Navigation and Control Design

For the robot to autonomously plan and cover the entire field, while making sure to avoid obstacles, the robot needs to plan a strategy to cover the entire field. The first section compares and analyzes the different path planning and coverage strategies that the robot can take to cover the field. The next section then analyzes how the control design architecture of most autonomous robots works.

2.3.1 Field Coverage

In agriculture, Coverage Path Planning (CPP) algorithms must prioritize adaptability and real-time decision-making. Agricultural robots, such as autonomous tractors or drones, operate in large outdoor environments with uneven terrain and unpredictable obstacles such as plants, animals, and terrain variations. The terrain, while mostly static, presents temporary obstacles (pests, human workers, animals) that the robot must detect and work around[15].

Most CPP algorithms use cellular decomposition, which divides a known environment into smaller units such as grid squares. Many algorithms also employ occupancy grids, in which each cell is designated as free, occupied, or unknown. As the robot moves through the environment, it updates the grid in real time based on sensor data, keeping its internal model accurate[15].

Table 2.2 compares different CPP strategies based on complexity, space required, coverage certainty, mode, and best use case. "Mode" here refers to whether the robot needs to be online or not for the computation of the planned path. For an agricultural robot, typically a strategy with the most coverage, least space and time complexity, and offline mode would be most suitable to cover the field. Based on the table, the most suitable algorithms are Boustrophedon, Trapezoidal, and Particle Swarm Optimization (in the case of multi-robots)[15].

2.3.2 Control Design

The control structure for a robot depends heavily on its operating environment, the tasks it must perform, and its safety requirements. A robot in a healthcare facility, for example, has very different reaction-time, obstacle-avoidance, and task-execution requirements than an agricultural robot.

Several strategies exist for planning the control structure for a robot. Some of those are described in Table 2.3. Out of these, the most popular one used in agricultural robotics is the Finite State Machine (FSM). It decomposes robot behavior into a discrete, well-defined set of states with explicit rules governing transitions between them based on sensory inputs

Method	Complexity	Coverage Certainty	Storage Required	Mode
Grid-Based / Lawnmower (Boustrophedon)	Low - O(n)	Very High	Low (Memory scales linearly with area)	Offline
Spanning Tree Coverage (STC / Graph-Based)	Medium Building spanning tree over mega-cells is O(n log n). Path traversal is O(n).	High (partial obstacles in mega-cells may cause gaps)	Medium (Memory scales with number of nodes and edges)	Both (Offline preferred; Online extensions exist)
Cellular Decomposition (Trapezoidal / Approximate)	Medium decomposition is O(n log n) coverage path within cells O(n).	High	Medium (Scales with obstacle complexity)	Offline
Sampling-Based (PRM / RRT)	High PRM roadmap construction is $O(n^2)$ RRT is O(n log n) on average but unpredictable in cluttered spaces.	Probabilistic completeness only (coverage is not guaranteed)	High	RRT: Online PRM: Offline
Wavefront / BFS-Based	Medium - O(n)	High	Medium (Memory proportional to grid size)	Offline
A* / D* / Greedy Search	Medium-High A* is O(n log n) per query; D* adds replanning overhead for dynamic changes.	Medium-High coverage depends on integration with CPP planner (used for path segments rather than area coverage)	Medium	A*: Offline D*: Online
Genetic Algorithm (GA / Meta-Heuristic)	Very High O(g × p × f) where g = generations p = population f = fitness evaluation cost	Medium (full coverage not guaranteed)	High (Memory scales with population size and path encoding length)	Offline
Particle Swarm Optimization (PSO) / Swarm	High O(i × n × d) where i = iterations n = particles d = dimensions Faster than GA but still costly.	Medium (local minima may cause incomplete coverage)	Medium-High (Scales with swarm size and number of iterations)	Online (adaptive variant)
Reinforcement Learning (Q-Learning / DQN / PPO)	Very High training phase is extremely costly - (O(episodes × steps × state-action space)) Inference is fast.	Medium (policy-dependent)	High (training) / Low (deployment)	Online (inference) Offline (training)
Random Walk / Stochastic	Very Low O(1) per step decision No planning overhead	Low (No guarantee of full coverage)	Very Low (only needs current position and direction)	Online

Table 2.2: *Comparison between CPP strategies*

Architecture	Advantages	Disadvantages
Finite State Machine (FSM)	Transparent, deterministic, O(1) state evaluation. Each state directly represents a physical robot behaviour. Easy to debug via state logs. Safe failure modes by design.	State explosion if requirements grow significantly. No learning or adaptation between missions.
Hierarchical FSM (HFSM)	Groups states into sub-machines, reducing complexity in large behaviour sets. Natural upgrade path from FSM.	More complex to implement and debug across hierarchy levels.
Behaviour Trees (BT)	Highly modular and reusable. Widely adopted in ROS2 Nav2 and modern field robots. Easy to add/remove behaviours.	More complex than FSM; reactive nature not always intuitive for sequential missions.
Reactive / Subsumption	Ultra-lightweight; no state memory required. Immediate response to sensor inputs.	No global plan or memory. Cannot handle tasks requiring sequencing.
Reinforcement Learning Policy	Can learn optimal policies without explicit programming. Adapts to novel situations over time.	Requires training, reward shaping, and large compute. Black-box — hard to certify for safety-critical outdoor machinery.
PDDL / HTN Task Planning	Supports complex goal decomposition over long horizons. Formally verifiable in principle.	Very high computational cost. Requires complete domain model. Overkill for a single robot on a structured field.

Table 2.3: *Comparison between different control strategies[15], [16]*

or internal conditions[16].

FSMs offer several advantages that make them well suited for agricultural robots. They are computationally cheap—state evaluation and transition checking are both O(1) operations—which is critical for low-power microcontrollers. They are fully predictable: a finite set of inputs maps to a finite set of outputs, making the control structure easy to debug and reason about. They are easy to extend, since new behaviors require only adding new states and transition conditions. Finally, fault and emergency states can be defined with guaranteed safe-stop transitions, supporting safe operation[15], [16].

2.4 Hardware Design Influences

Two hardware design choices from existing agricultural robots were identified as particularly suitable for inclusion in this prototype: an LED/photoresistor probe-insertion feedback system and a passive wheel-leg transformable mechanism.

2.4.1 LED/Photoresistor Feedback for Autonomous Probe Insertion Depth[3]

Linford and Haghshenas-Jaryani describe a closed-loop optical feedback system using a paired LED and photoresistor to determine whether a soil moisture probe has been inserted to the correct depth—i.e., fully flush with the soil surface. The system is mounted on a custom 3D-printed end-effector housing at the tip of a 6-DoF robotic manipulator arm, which is itself mounted on a 4-wheeled rover.

The challenge this addresses is fundamental: a soil moisture sensor must be inserted to a precise depth to give accurate readings. If the probe is not flush with the soil surface, the measurement is unreliable. On uneven, rough agricultural terrain, using only joint-angle-based position control is insufficient—the actual soil surface height varies, and the ground slope changes along the row.

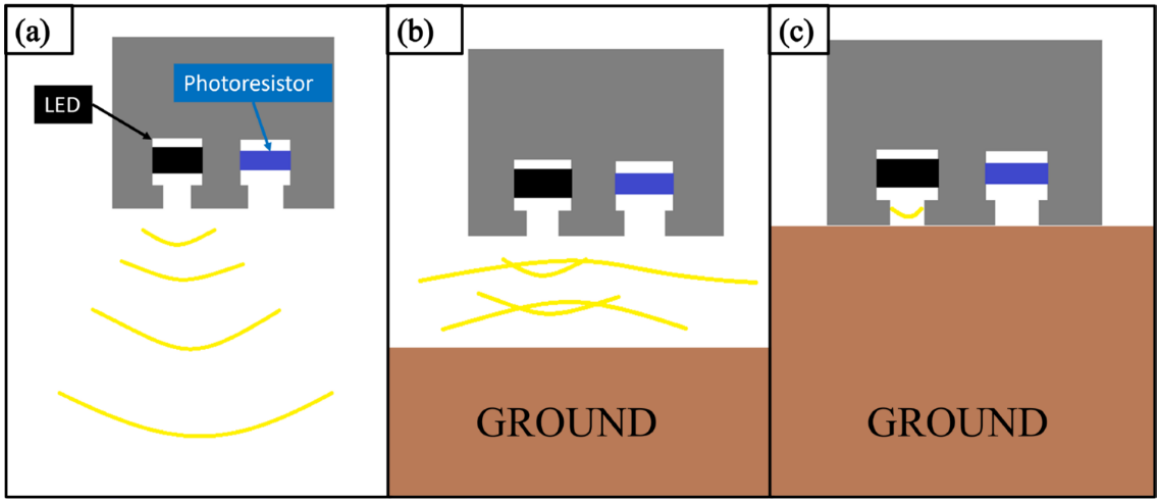


Figure 2.1: *Photoresistor feedback schematic*

The feedback mechanism operates on a simple optical principle:

a Far from soil, the photoresistor reads only ambient light. The LED has no effect at this distance.

b Approaching soil (25 mm away): the LED illuminates the soil surface, and the light

reflected back to the photoresistor is at its maximum—the LED's output partially reflects off the surface into the photoresistor.

c Flush with soil (0 mm): the geometry of the tool housing blocks all light paths. The photoresistor reads its minimum value, indicating complete insertion.

2.4.2 Passive Geared Wheel-Leg Transformable Mechanism[17]

Zheng and Lee present the WheeLeR mechanism: a passive wheel-leg transformable system that allows a single driven wheel to switch between a conventional circular rolling configuration and a multi-legged walking configuration without requiring a separate actuator for the transformation.

The motivation is a fundamental trade-off in mobile robotics: wheeled locomotion is fast and energy-efficient on flat, smooth ground but performs poorly on uneven terrain, loose soil, sand, gravel, and obstacles. Legged locomotion handles these conditions far better but is slower and more mechanically complex. The WheeLeR mechanism allows a single robot to exploit both modes adaptively.

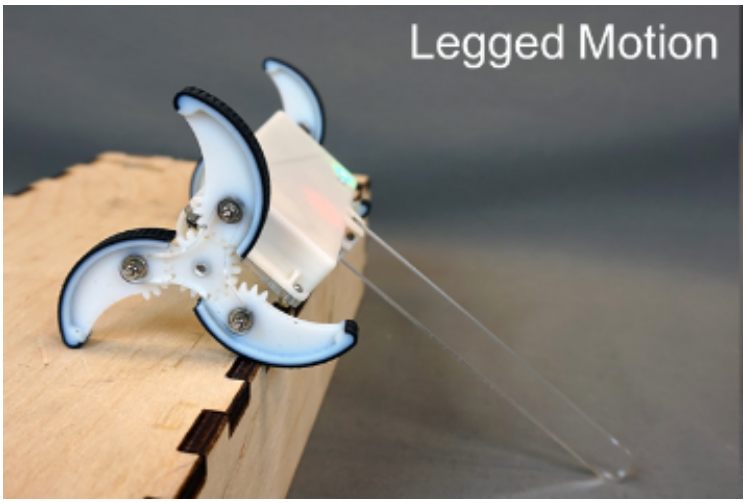


Figure 2.2: *Wheel that can traverse through rocky terrain*

Chapter 3

DESIGN

The proposed robot is a compact autonomous ground platform for crop monitoring and leaf disease detection. Its hardware comprises a 25 cm × 25 cm body, 4 spiked wheels to traverse tilled soil better, a front-facing camera, IR sensors for obstacle detection, a light sensor, temperature and humidity sensors, an inertial measurement unit, drive motors and a soil moisture probe mechanism. The software handles row coverage, finite-state control, on-device disease detection, local logging, data upload during charging, and a frontend dashboard that visualizes coverage and field measurements.

The main objectives of the autonomous robot are:

- To move autonomously within the predefined boundaries of the field, starting from an edge.
- To map the field, detect obstacles, avoid them, and mark them on its internal map.
- To cover the entire field (excluding obstacles) and return to the docking station on its own.
- To periodically sample crops and soil, process and store findings, and upload them to Firestore.
- To detect low battery and return to the docking station to recharge.
- To present findings in a way that is easy for the user to view and interpret through a frontend application.

To make these objectives tractable within the scope of a prototype, the following constraints were applied:

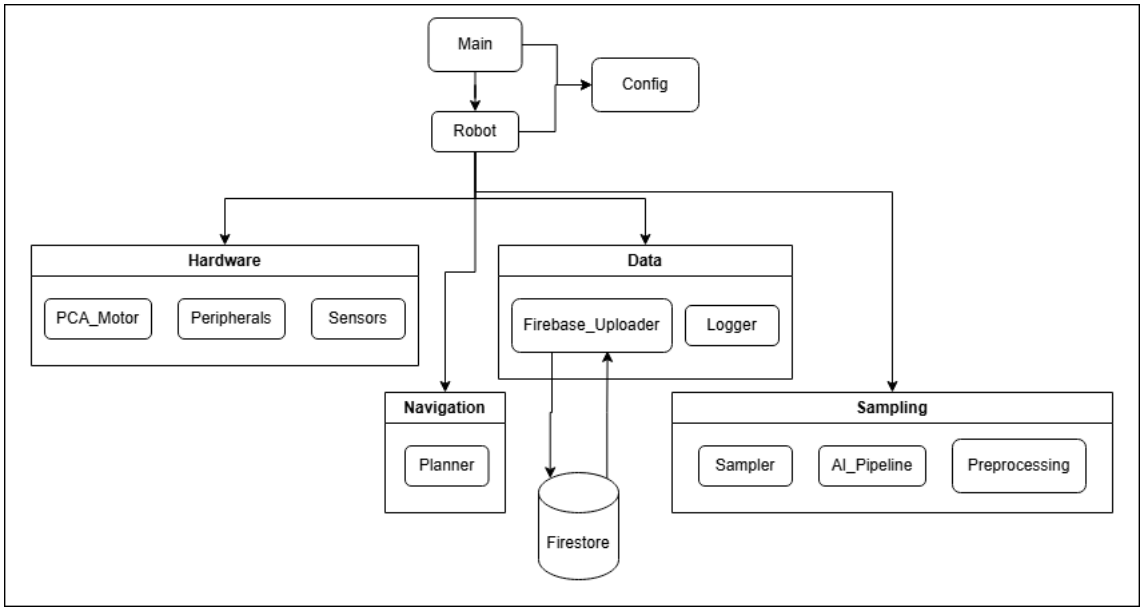


Figure 3.1: *Class Diagram*

- The robot will start at the docking station, which will be located at the bottom-right point of the field, such that the crops are planted parallel to the edge of this field.
- The field dimensions will be hard-coded in the software.

The designs discussed were first tested in Webots[18], a simulation software, where a rough draft of the robot was built and tested on an uneven, field-like terrain. Only the software and hardware part were tested in the simulation, without any AI processing. Once acceptable results were seen in the simulation, physical designs and related code were made and implemented.

3.1 Software Design

To keep the software readable and maintainable, the codebase was split into five logical modules (Figure 3.1): main (configuration and run), hardware, navigation, sampling, and data. Each module is responsible only for its own functionality, keeping concerns separated.

The dashboard is a separate application that never communicates directly with the

robot; both components interact only through Firestore. While this introduces some latency between robot activity and dashboard updates, it allows the dashboard to remain available to the user even when the robot is outside network coverage.

3.1.1 Robot States

The main control logic runs as a single continuous loop. Each iteration is called a tick. On each tick, the robot either remains in its current state or, if transition conditions are met, moves to a new state. Each state has its own responsibilities that must be completed before transitioning.

As shown in Figure 3.2, the robot begins in the INIT state, which initializes hardware and loads any previously stored grid. It then transitions to GO_TO_START, navigating from the docking station to the edge of the field. In the MOWING state, it traverses the field following a boustrophedon pattern. Every 30 seconds, it enters the SAMPLING state to record soil and plant parameters, then resumes mowing. If the IR sensors detect an obstacle, the robot enters AVOIDING until the obstacle is cleared and the affected grid cell is marked, then resumes mowing. If the wheels stop moving despite motor commands, the robot enters STUCK and executes a recovery maneuver before resuming. When the battery is low or the field is fully covered, the robot returns to the dock to charge; once charged, it restarts or resumes the run. A summary of each state is given in Table 3.1.

Nearly all data collection occurs during the SAMPLING state, which follows a fixed sequence: capture leaf images, then record soil and environmental parameters. The camera first photographs the leaves on the left, then the leaves on the right, then returns to its starting position. The soil probe is lowered into the ground, with an LED/photoresistor pair confirming that it is flush with the soil surface. The probe records temperature, humidity, and moisture level, and is then retracted. Finally, the captured images are processed in two steps: a simple heuristic determines whether the image contains a leaf, and only candidate leaf images are passed to the disease-detection model. The heuristic checks for a high ratio of green pixels, high saturation, and a moderate edge density—higher than a uniform background like open sky, but lower than highly fragmented scenes.—to reduce

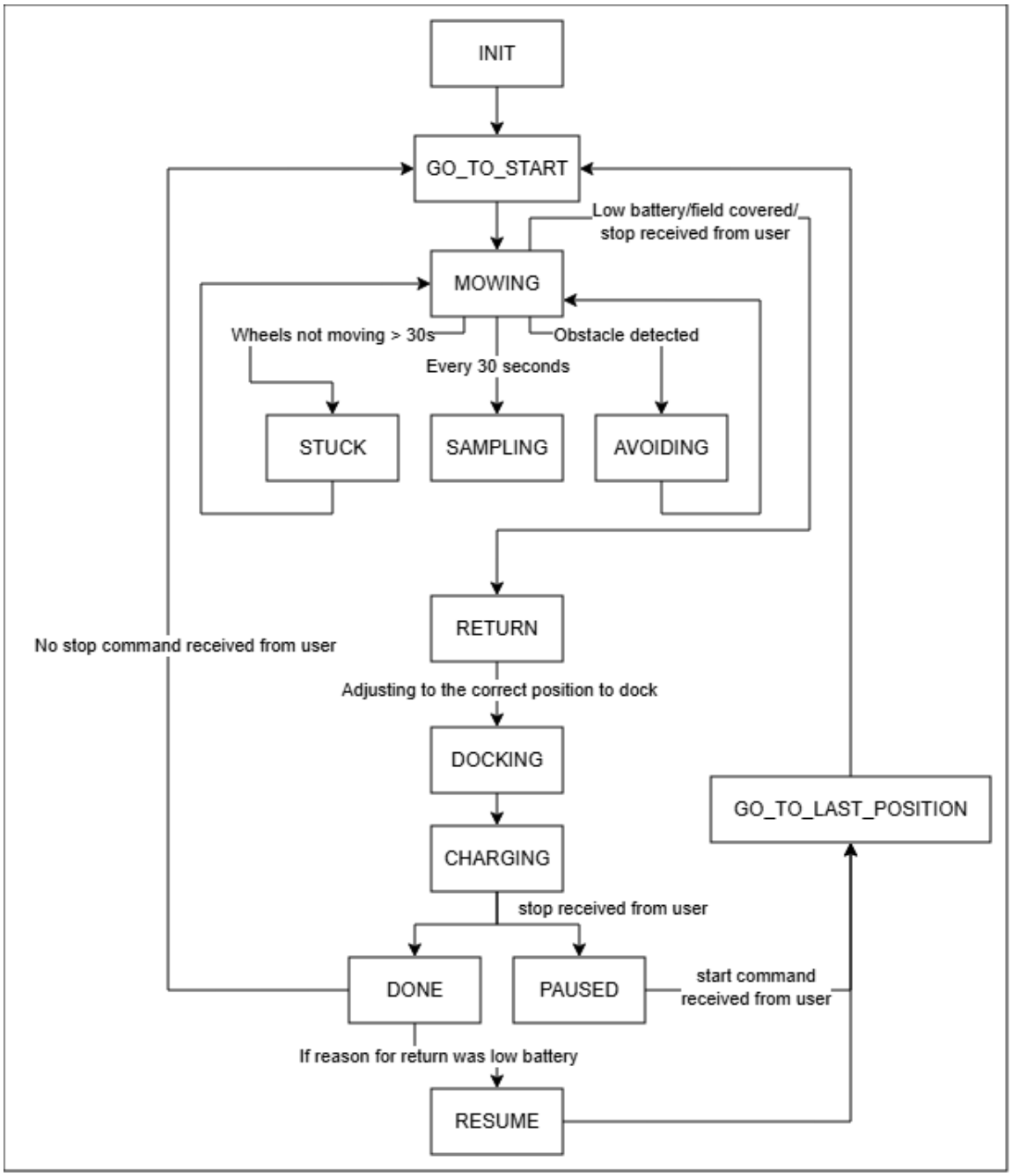


Figure 3.2: *State Diagram for Robot*

State	Description	Exits when
INIT	Hardware initialised, I^2C bus opened, grid loaded from local storage if exists	Immediately → GO_TO_START
GO_TO_START	Dijkstra path planned from dock to field start position. Robot follows waypoints.	Start position reached → MOWING
MOWING	Follows boustrophedon (lawnmower) pattern. Marks cells VISITED. Polls IR every tick. Polls FireStore command every 10 s.	30 s elapsed → SAMPLING; IR/bump → AVOIDING; encoder stall → STUCK; battery low → RETURN; dashboard stop → RETURN
SAMPLING	Blocking. Runs the full sampling sequence (env sensors → camera → soil probe → AI). Returns SampleRecord.	Sequence complete → MOWING
AVOIDING	IR or accelerometer bump triggered. Robot backs up and curves around obstacle. Marks obstacle cell on grid.	Obstacle clear and hard deadline not exceeded → MOWING; hard deadline exceeded → MOWING (force exit)
STUCK	Encoders report zero movement despite non-zero motor duty (wheels spinning in place or genuinely blocked).	Randomised escape manoeuvre complete → MOWING
RETURN	Battery below threshold, or dashboard Stop command received. Dijkstra plans path back to dock.	Dock position reached → DOCKING
DOCKING	Robot reverses slowly into dock till charging is detected.	Detected charging → CHARGING; if dashboard-stop: → PAUSED
CHARGING	Motors off. Uploads all session data to Firestore. Deletes local images and session information.	Upload complete → DONE
DONE	Run complete. Goes to GO_TO_START state.	Immediately → GO_TO_START
PAUSED	Human-commanded stop. Motors off. Polls Firestore every 10 s for Start command.	Dashboard "start" received → GO_TO_START

Table 3.1: *Description of each state of the robot*

unnecessary processing on non-leaf images. After image processing, the robot transitions from SAMPLING back to MOWING.

3.1.2 User Dashboard

The user dashboard is a single HTML page served by a minimal Flask app. The Flask server does no data processing — it only serves the HTML file. All data comes directly from Firestore via the Firebase JavaScript SDK running in the browser. The dashboard is accessible from any device with internet access.

On first load, a modal dialog prompts for the Firebase web app config JSON. This is stored in localStorage—the dialog does not appear on subsequent visits. After the modal dialog is dismissed, each subsequent session displays the following components: header bar, session sidebar, field coverage map, disease summary panel, sensor charts, sample gallery, and the sample detail modal. More details about each section can be found in Table 3.2.

The dashboard includes a STOP/START control in the header bar. The user can issue a STOP command to interrupt a field run and return the robot to its docking station—useful in severe weather, when the user wants to work in the field themselves, or for any other reason. Once stopped, the robot resumes only when the user issues a START command from the dashboard. Note that the robot can only receive these commands when it is within network coverage.

3.1.3 Firestore

The data stored in Firestore follows the structure shown in Figure 3.3. Within the robots collection, all data is grouped under the document fieldbot-01, the unique identifier for the individual robot. Organizing data by robot ID makes the design easily expandable to multiple robots. Each robot contains four sub-collections—commands, grid, sessions, and status—described below:

- **commands:** Stores the information needed to issue user-defined start/stop instructions, including the command type, whether an acknowledgment was received, and the time the command was sent.

Section	Description
Header bar	Always visible, updates in real-time. Houses robot parameters such as session count, total samples collected across all sessions, total diseased samples collected in current session, live status of the robot. Has robot stop/start button to remotely control the robot.
Session Sidebar	Always visible. Lists all the sessions covered by the robot, and session details such as date, sample count and coverage percentage.
Field Coverage map	Visible on session selection. Shows the field covered along with different cells coloured in different colours showing their status - visited, unknown, obstacle, sample location, live location.
Disease Summary Panel	Visible on session selection if samples collected. Gives an overview of diseased plants found in the current session and their position.
Sensor Charts	Visible on session selection if samples collected. Shows the variance of temperature, humidity and soil moisture level with time.
Sample gallery	Visible on session selection if sample collected. Shows one card per sample. When clicked to open, shows the image, temperature, humidity, soil moisture sensor values, disease detection result along with the position where sample was taken.

Table 3.2: *Description of user dashboard application sections*

- **grid:** Stores all grid-related information, including the field map, obstacle positions, decay times, coverage percentage, and the ID of the most recent session associated with the grid.

- **sessions:** Stores the data for each individual run. Every session has a unique ID under which it records the number of samples collected, the percentage of the field covered, sample details, any diseases detected, and the session's start and end times.

- **status:** Tracks the robot's live status, updated approximately every 10 seconds. It stores the current coverage percentage, battery percentage, position, state, and sample count.

3.1.4 Key Design Decisions

Several design decisions were taken to make sure the software for the robot was simple and easy to implement and maintain, as well as to ensure the system remained fault tolerant. These are described below:

- The AVOIDING state has a hard deadline to prevent the robot from getting stuck in it. Without this safeguard, the robot might never complete its field coverage objective.

- The STUCK state uses a randomized escape route instead of a deterministic one, making sure that there is a higher probability of the robot getting unstuck while still keeping the code simple.

- During the field run, all data is written in local storage. The robot has no network dependency during this operation. Upload only happens at the dock. This means a Wi-Fi dropout in the middle of a field run loses no essential data — the upload just happens later.

- The user dashboard application talks directly to Firebase. The robot does not need to be reachable, on the same network, or even online during the field run. This makes sure that the user is able to see the data collected and the last known state of the robot, even if the robot is out of the network.

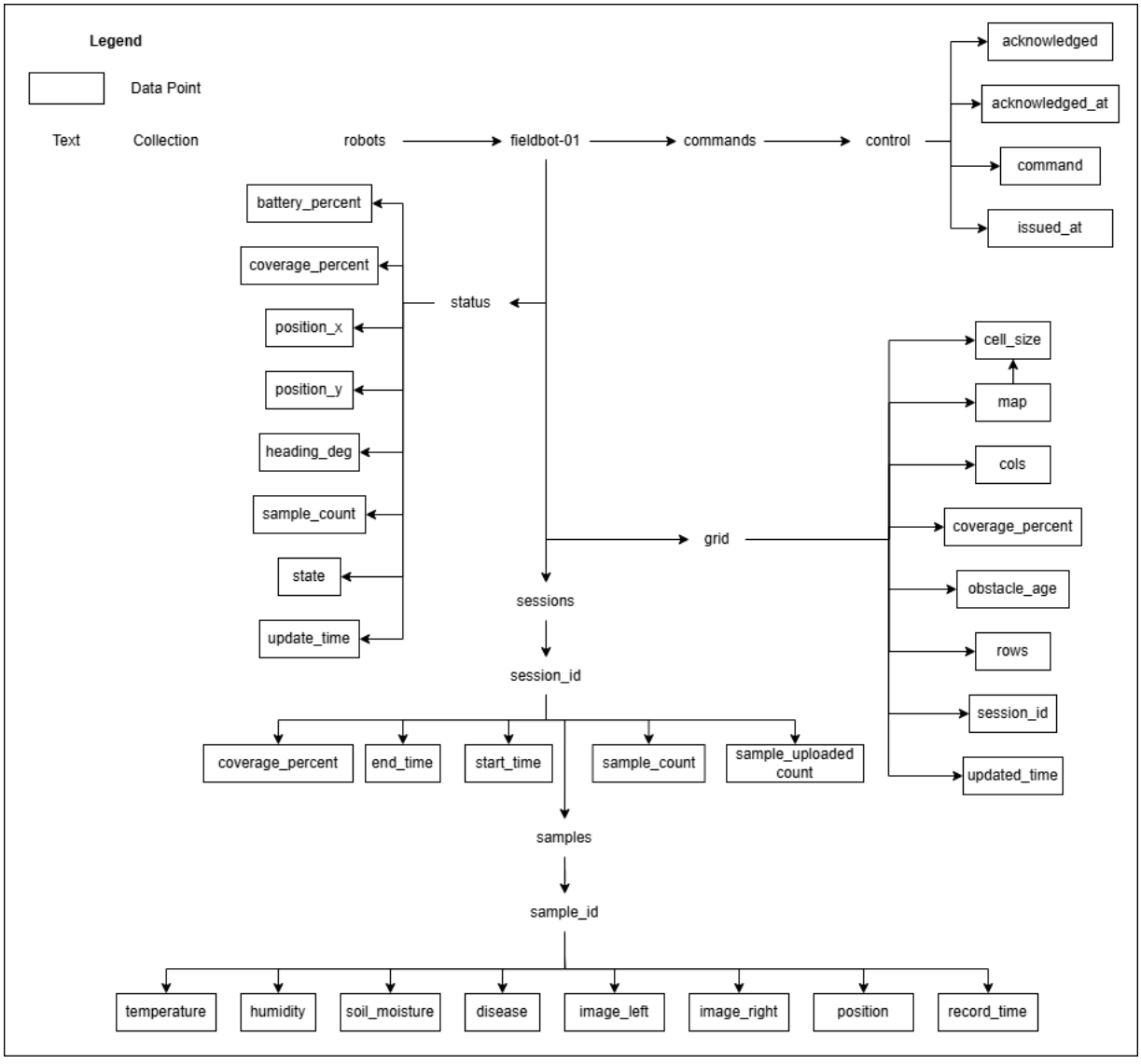


Figure 3.3: *Firebase Data Storage Design*

- The robot sends out status updates every 10s, which is on a daemon thread, preventing blocking the control loop (in case of timeouts or errors).
- The stored grid uses a time-based expiry so that obstacle markings do not persist indefinitely. This prevents transient obstacles (such as animals or temporarily parked equipment) from being permanently marked, and allows the grid to adapt to seasonal crop layout changes.
- All pins, thresholds, and field dimensions are kept in the config file. No constants scattered across files. For example, changing the field size requires editing two variables.

3.2 Hardware Design

3.2.1 Electronics

For the complete functionality of the prototype, a number of hardware components were required, the details of which are given below:

- Raspberry Pi 5
- AI Hat (for better processing power)
- Active Cooler
- Raspberry Pi Camera Module 3
- SparkFun Qwiic/STEMMA QT multiplexer
- Si7021 Temperature & Humidity Sensor
- TSL2561 Luminosity Sensor
- RGB LED
- STEMMA Soil Sensor - I^2C Capacitive Moisture Sensor
- PCA9685
- TB6612FNG Motor Driver

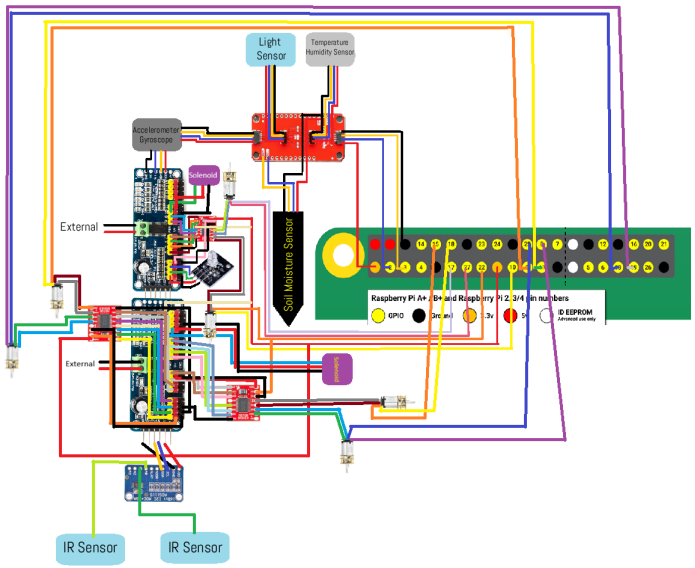


Figure 3.4: *Electronic Schematic diagram*

- N20 Encoder Motor
- Relay Module
- Solenoid Latch
- ADS1015 Analog to Digital Board
- IR Sensor
- Battery Hat

The connections between different components are given in Figure 3.4. The physical hardware is split into four subsystems: drive, mechanisms (soil probe + camera arm), sensing, and power. All motor and solenoid control is handled by two PCA9685 PWM controllers connected to the Pi over I^2C, and all environmental sensors share the same I^2C bus.

3.2.2 Body Design

The outer body was designed with reference to existing agricultural robots, balancing component dimensions against required functionality. A main outer enclosure houses the sensors, actuators, Raspberry Pi, and battery hat. A smaller box, inspired by Linford and Haghshenas-Jaryani[3], holds the soil moisture probe, temperature and humidity sensor, LED, and light sensor; this box attaches to the main body via a motor that lowers it into the soil during sampling. The wheel design draws on the WheeLeR mechanism[17], but uses a fixed geometry rather than a transformable one to keep the overall design simple.

3.3 AI Experiment Design

To support real-time on-device disease inference, the model must classify plant diseases accurately, run quickly, and remain lightweight so that most of the device's storage can be reserved for collected samples rather than the model itself. To identify the most suitable model, each candidate was trained on a subset of the PlantVillage dataset[19], an open-access repository of 54,306 images of healthy and diseased plant leaves covering 14 crop species and 26 diseases, making it one of the largest publicly available datasets for computer vision in agriculture. The subset used here covers tomato, potato, and bell pepper crops across nine diseases, totaling approximately 20,600 images.

Model selection followed a four-stage pipeline: image preprocessing, feature extraction, model training (with and without pruning), and comparative evaluation.

3.3.1 Preprocessing

Each image in the dataset went through the same preprocessing steps, which involved:

1. Resizing image to 160×160 pixels
2. Applying a cellular automaton filter for noise removal from images
3. Applying histogram equalization for brightness normalization
4. Applying Wiener filtering to further deblur the image and remove noise
5. Removing the background of the image using a Gaussian filter

3.3.2 Feature Extraction

For each of the preprocessed images, inspired by Vijayan & Chowdhary (2025)[11], 143,000+ features were extracted and stored for further processing. These features included:

- Color moments, that is, mean, skewness, standard deviation, and kurtosis of each color, to capture the statistical distribution of pixel intensity across color channels. These are compact, rotation-invariant, and robust to illumination changes, making them well-suited for distinguishing disease-caused discoloration (yellowing, browning, and lesions) from healthy green tissue. These were computed per channel across 5 color spaces (RGB, LAB, HSI, HSV, and LUV), giving 5 spaces $\times$ 3 channels $\times$ 4 statistics = 60 color features in total.

- Gray Level Co-occurrence Matrix (GLCM) descriptors. GLCM is a second-order statistical method that considers the spatial relationships between pairs of pixels. It builds a matrix counting how often pairs of pixel intensities appear at a given offset (distance and angle) from each other. From this matrix, five key descriptors were extracted:

  - **Angular Second Moment (ASM):** Sum of squared GLCM entries, which measures textural uniformity. Healthy leaves have higher uniformity than diseased ones.
  - **Contrast:** Measures local intensity variation between a pixel and its neighbor. Diseased leaves have higher contrast.
  - **Correlation:** Linear dependency of grey levels across the image. As this quantifies how predictably pixel values relate to neighbors, it is disrupted by disease spots.
  - **Entropy:** High entropy means complex, irregular texture typical of fungal or bacterial spread patterns.
  - **Inverse Difference Moment (IDM):** This measures closeness of the distribution to the diagonal, that is, local homogeneity. Low IDM indicates heterogeneous texture, characteristic of spotted or blighted regions.

- Higher-order statistical features, derived from clusters of continuous pixels with similar gray levels, captured using the gray-level run length matrix (GLRLM). A "run" is a sequence of consecutive pixels with the same grey level in a particular direction. The GLRLM counts how many runs of each length exist for each grey level. Key texture features derived from this in the paper were the following:

  - **Short Run Emphasis (SRE):** High value indicates many short runs, i.e., fine texture (e.g., early-stage lesions).
  - **Long Run Emphasis (LRE):** High value indicates coarse, elongated texture regions (e.g., streak diseases like leaf blast).
  - **Run Length Non-uniformity (RLNU):** Measures variation in run lengths. High RLNU means diverse texture across the leaf.
  - **Grey Level Non-uniformity (GLNU):** Measures the spread of runs across gray levels, which capture tonal variety in diseased tissue.

- Gray Level Dependence Matrix (GLDM) features, which extracts texture features by calculating the absolute difference in gray levels between two pixels separated by a specified distance. Where GLCM looks at the co-occurrence of exact values, GLDM focuses on how dependent neighboring pixels are—i.e., how similar a pixel is to its neighbors regardless of direction. Key features included:

  - **Small Dependence Emphasis:** Favors regions where pixels differ greatly from neighbors (lesion edges).
  - **Large Dependence Emphasis:** Favors regions where pixels are highly similar to neighbors (uniform healthy tissue).
  - **Dependence Non-uniformity:** Variation in neighborhood dependence across the image.

- Local Binary Pattern (LBP) texture features. LBP employs a shape-based operator to threshold neighboring pixels based on the intensity of the central pixel. Each pixel is compared to its surrounding neighbors—if the neighbor is brighter, a 1 is assigned;

if darker, a 0. This produces a binary code per pixel that encodes local micro-texture. LBP is particularly valuable in capturing these texture features:

  - Capturing fine-grained surface texture (fungal spore patterns, rust pustules).
  - Being invariant to monotonic grey-level changes (robust to field lighting variation).
  - Distinguishing between disease types that share similar colors but differ in surface texture.

- Histogram of Oriented Gradients (HOG) which focuses on the structural aspects of the image by analyzing gradient orientations within localized regions using a feature descriptor. The image was divided into small cells; within each cell, the gradient magnitude and direction of each pixel are computed and then histogrammed across 9 orientation bins. Adjacent cells are grouped into blocks and normalized for illumination robustness. It captures:

  - **Edge and boundary structure:** Sharp disease lesion borders have strong, consistent gradients.
  - **Shape of diseased regions:** Elongated streak patterns (leaf blast) vs. circular spots (brown spot) have distinct orientation histograms.
  - **Structural regularity:** healthy tissue has diffuse gradients; necrotic patches have concentrated edge directions.

- A number of shape features were also extracted from each image, which were area, perimeter, length/width, equivalent diameter, centroid, eccentricity, orientation, major/minor axis length, axis ratio, convexity, Hu invariant moments, and Zernike moments.

3.3.3 Best Feature Selector

After the previous step, three nature-inspired metaheuristic algorithms searched the 143,019-dimensional binary feature space to find the most discriminative subset. Each algorithm was

Selector	Mechanism	Parameters
ABC	Artificial Bee Colony - employed, onlooker, and scout bees forage for optimal feature subsets	10 bees, 20 iterations
WOA-APSO	Whale Optimisation + Adaptive Particle Swarm Optimization hybrid - combines whale bubble-net attack with particle swarm velocity updates	10 agents, 20 iterations
ACO	Ant Colony Optimisation - pheromone trails bias ants toward high-variance features; evaporation prevents stagnation	10 ants, 20 iterations

Table 3.3: *Description of Feature Selectors*

scored using a small dense proxy network (SimpleVGG16 in feature-input mode) as the fitness function. Features were pre-filtered to the top 1,000 by variance before the search, and the proxy used a fixed 32-dimensional random projection to keep GPU memory bounded. The details of the three feature selectors are given in Table 3.3.

3.3.4 Model Comparison

Five distinct architectures (six configurations counting Hybrid CNN in both modes) were trained and evaluated on the test set. The feature-input model used features selected by the best feature selector in the last step. All image-input models received the 160×160 preprocessed images directly. Each model was compared on the following metrics: accuracy, F1 macro, prediction time (in seconds), and model size (in MB). In addition, the training time and the FLOPs achieved were also calculated.

Image-Input Models

- **VGG16:** 16-layer deep CNN pretrained on ImageNet. Baseline model and fitness evaluator inside the feature selection proxy.
- **LiteCShuffle:** Lightweight channel-shuffle network. Grouped convolutions with shuf-

fling between groups (ShuffleNet-inspired) are supposed to give high accuracy at very low parameter count and FLOPs, inspired by Sangeeta et al. (2025) [13].

- **YangViT:** Biologically-inspired network modelling the visual cortex (V1). The winner-take-all lateral inhibition layer suppresses neighboring neurons, followed by a decision-making layer modeled on prefrontal cortex dynamics. Inspired by Yang et al.[10].
- **Hybrid CNN:** This model can intake both images and features, wherein raw images are taken directly by the CNN and processed to gather important information from the images[11].

Feature-Input Models

- **KCNet (Kenyon Cell Network):** Random fixed input weights (kernel trick) with only the output layer trained via ridge regression. Instantaneous training, no back-prop. Inspired by Hong and Pavlic (2025)[12]
- **Hybrid CNN:** Custom CNN combining depthwise separable convolutions with squeeze-and-excitation channel attention blocks, inspired by Vijayan and Chowdhary (2025)[11]. Lighter than VGG with adaptive channel recalibration.

Channel Pruning

Inspired by Liu and Zhao (2023), an experiment was run where each image-input Keras model with Conv2D layers (LiteCShuffle, YangVit and HybridCNN) was pruned by removing the lowest-magnitude filters (30% pruning ratio), then fine-tuned for 5 epochs. This reduces model size and FLOPs while preserving accuracy — critical for deployment on robot hardware with limited compute.

These are the steps taken for pruning each model:

1. Computing per-filter L1 norm across each Conv2D layer
2. Removing the bottom 30% of filters by magnitude
3. Reconstructing the model with reduced channel counts

4. Fine-tuning for 5 epochs on the training set

Chapter 4

EXPERIMENTAL RESULTS AND DISCUSSION

Assembling the components and running the experiments produced a 25 cm × 25 cm autonomous robot capable of covering ground independently, avoiding obstacles, and sampling soil and plants to record temperature, moisture, and disease indicators. The total prototype cost was under $1,000 USD. Detailed results for each subsystem are given in the subsections below.

4.1 Simulated Result

To validate the robot's functionality before physical fabrication, the robot body was first designed in Webots (Figure 4.1). The simulated design includes a camera with four degrees of motion—later reduced to two in the physical implementation to simplify mechanics—and two front-facing IR sensors (shown as red rays in the figure) for obstacle detection. The body itself was kept simple: a rectangular chassis with a weighted rear compartment representing the Raspberry Pi and battery, mounted on four wheels.

The simulated robot was first tested in a maze-like environment (Figure 4.2a) using minimal obstacle-avoidance logic—turning right whenever an obstacle was detected. Once basic traversal was working, the robot was placed on a flat field-like terrain to test the boustrophedon coverage pattern (Figure 4.2b). The yellow rectangle marks the field boundary and the red posts indicate its corners. The robot covered the full field in approximately ten minutes of simulated time, excluding sampling. Each sample added roughly one minute.

The simulation informed several design decisions, including the angular separation between the two IR sensors (80°), the height of the camera mast (approximately 25 cm), and the optimal start position. An alternative coverage strategy—Spiral STC with particle swarm optimization—was also tested but rejected in favor of boustrophedon, which was simpler to implement and better matched typical row-crop geometry. The simulation also

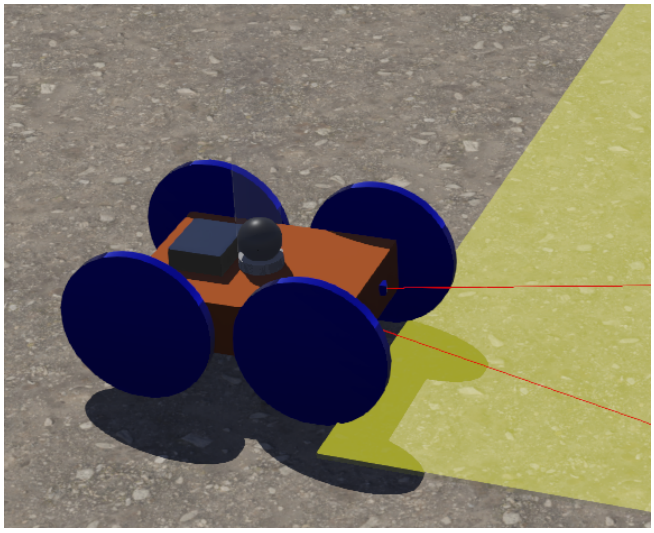
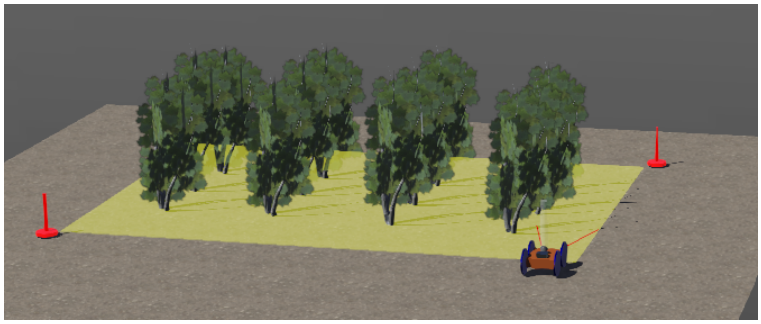


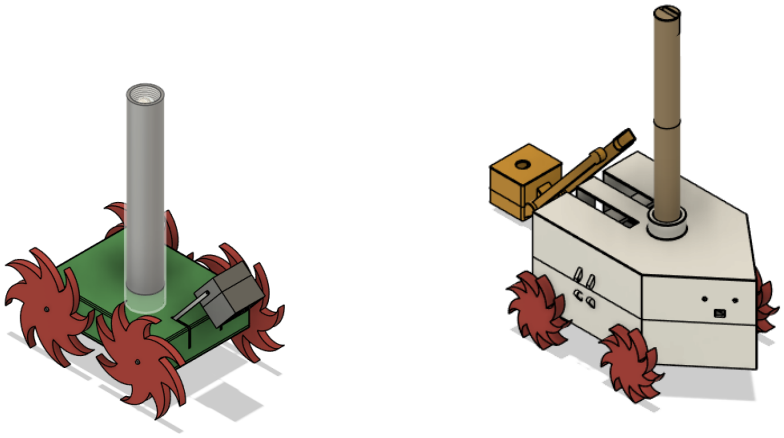
Figure 4.1: *Simulated Robot Design*

(a) Maze like field for initial robot to traverse through



(b) Realistic field for Boustrophedon coverage

Figure 4.2: *Simulated fields for the robot to traverse through*



(a) First Assembly Design for Complete Robot

(b) Final Assembly Design for Complete Robot

Figure 4.3: *Assembly Design Evolution*

revealed that odometry-based position tracking alone produces significant drift unless the robot has sufficient mass to maintain consistent wheel contact. To address this, the final design includes an accelerometer and gyroscope to improve position estimation.

4.2 3D Prints

The robot body was iterated through several physical prototypes, adjusting for component dimensions and assembly constraints. The first 3D-printed design (Figure 4.3a) closely mirrored the simulated robot. Subsequent revisions added a fixed angular mount for the two IR sensors, a latch system to split the robot into two equal halves for easy access, a port for the charger cable, smaller wheels to prevent inter-wheel collision given the body size, and protruding wheel-arches to better accommodate internal wiring.

The soil-probe enclosure (Figure 4.4) includes recessed cavities for the LED and light sensor. The wheels (visible in Figure 4.3) use a serrated profile designed to traverse small rocks and uneven soil. The camera is mounted atop the vertical mast shown in Figure 4.3b, with a motor at its base for rotation, and the two IR sensors are screwed to the front of the chassis.

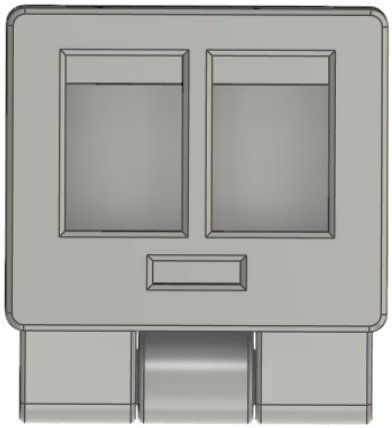


Figure 4.4: *Bottom of Soil Moisture Probe Box*

Selector	Proxy Accuracy	Features Selected	Selection Time[1]	Reduction
ABC	13.00%	492 / 143,019	513.3s (8.6 minutes)	99.66%
WOA-APSO	14.80%	458 / 143,019	255.5s (4.3 minutes)	99.68%
ACO	14.80%	1 / 143,019	245.0s (4.1 minutes)	99.99%

Table 4.1: *Results of each feature selector when used with a simple VGG16*

4.3 AI Experimental Result

4.3.1 Experiment 1 - Best Feature Selector

All three feature selectors yielded similarly low accuracy when paired with the VGG16 proxy model, but other characteristics differentiate them. ACO selected only a single feature, which is impractical for a real classifier. The best overall selector was WOA-APSO, narrowly outperforming ABC: it selected 458 features—a substantial reduction from the 1,000 highest-variance candidates that were given to each selector (the full 143,019-feature space was pre-filtered for tractability).

[1]s denotes seconds

Model	Accuracy	F1 Macro	Prediction Time[2] (in seconds)	FLOPs (in millions)	Train Time (in minutes)	Model Size (in MB)	Input
LiteCShuffle	97.63%	97.39%	8.9	1,052.8	52.8	1.15	Image
VGG16	93.97%	92.72%	35.2	134.8	73.3	827	Image
HybridCNN	91.06%	89.89%	6.5	5284.9	37.5	1.56	Image
YangViT	69.96%	60.88%	5.9	717.2	12.5	0.35	Image
HybridCNN	47.04%	38.37%	0.3	0.3	0.3	1.89	Features
KCNet	38.95%	24.97%	0.07	N/A	0.1	3.6	Features

Table 4.2: *AI Model Results (Without pruning models)*

4.3.2 Experiment 2a - Model Comparison

Most of the trained models were under 2 MB in size and therefore similarly suitable for deployment on the robot; the exception was VGG16, which was retained only as a baseline since most computer-vision tasks use either VGG16 or VGG19[20]. Table 4.2 lists the five models in descending order of accuracy.

All image-input models significantly outperformed the feature-input models, likely because learned convolutional features capture disease-relevant structure more effectively than handcrafted statistical features in this dataset. This was reinforced by the Hybrid CNN result: when given both raw images and handcrafted features in parallel, the model relied predominantly on the raw-image branch.

All bio-inspired models tested also underperformed relative to traditional computer-vision architectures. A plausible explanation is that evolutionary heuristics in biology are typically specialized to a specific function, environment, and organism: these models may be effective for tasks other than leaf-disease classification, or better suited to crops not included in this study. Hybrid CNN, for instance, was originally evaluated only on rice-crop diseases and may not generalize beyond that domain. Similarly, YangViT was originally designed for gesture recognition and KCNet for binary odor classification—neither task closely resembles multi-class leaf-disease classification. More research is needed to determine when nature-inspired models can be successfully adapted to new domains.

[2]Time taken to predict if a leaf is diseased or not

Model	Pre-Prune Accuracy	Post-Prune Accuracy	Accuracy Change	F1 Macro (pruned)
HybridCNN	91.06%	88.69%	-2.37%	87.65%
LiteCShuffle	97.63%	91.35%	-6.27%	90.85%
YangViT	69.96%	56.35%	-13.61%	46.98%

Table 4.3: *Comparison of pruned vs unpruned AI models*

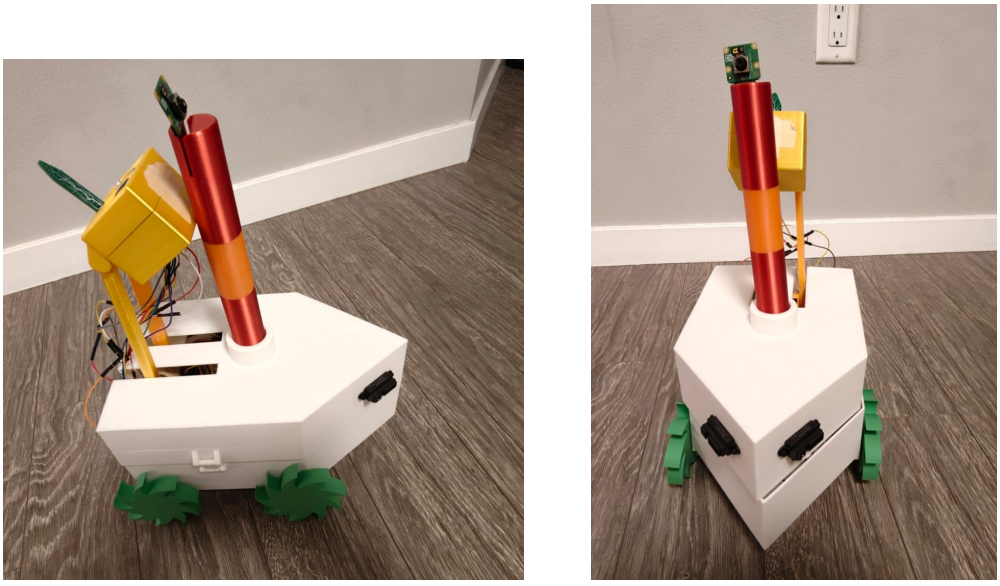
4.3.3 Experiment 2b - Channel Pruning

Automated channel pruning was performed on three models—-LiteCShuffleNet, HybridCNN, and YangVit—to see if they performed better. The results can be seen in Table 4.3. Pruning did not improve performance for any of the three models. The most plausible explanation is that the removed filters carried information important to classification, and since these architectures were already designed to be lightweight, there was little redundancy left to prune. This is consistent with the observation that channel pruning is most beneficial when applied to over-parameterized models.

Unpruned LiteCShuffle was selected as the deployment model: it achieved the highest accuracy and F1 score, returned predictions quickly, and had lower FLOPs than several alternatives. While not the fastest in inference time or lowest in FLOPs overall, both metrics were within acceptable bounds for the prototype.

4.4 Real World Assembly

The assembled robot is shown in Figure 4.5. It measures 21.5 cm in width and 25 cm in length and weighs 1.2 kg. The robot traverses a flat 1 m $\times$ 1 m surface in approximately 3 minutes and 50 seconds, stopping every 30 seconds to take a sample, for an average of three samples per run. Each sampling routine takes about 40 seconds, including image processing. Of this, the camera arm rotation and image capture account for roughly 20 seconds—5 seconds to rotate the arm and around 15 seconds for auto-focus and capture—while the soil probe takes 5 seconds to lower and 15 seconds to return to its upright position. Recording the soil parameters (temperature, moisture level, and humidity) takes less than a second. The soil moisture sensor is shown in Figure 4.6 while the lowered soil moisture box is shown



(a) Right Profile of Robot

(b) Front Profile of Robot

Figure 4.5: *Profiles of the assembled robot*

in Figure 4.7. Sensor accuracy depends on the specific components listed in Section 3.2, which were found to be reasonably accurate during trial runs.

Averaged across five runs, the robot consumes approximately 10% of its battery per 1 m $\times$ 1 m run, and a full recharge takes about one hour. Uploading images to Firebase introduces minimal lag, and sampling results are displayed on the dashboard. Maintenance is straightforward: the body can be opened using the latches on its sides, allowing a broken sensor or actuator to be swapped out with ease. Some components are more difficult to replace because they are glued to other parts—such as the wheel motors fixed to the body or the camera arm bonded to the camera-housing cylinder—but even these can be removed by applying isopropyl alcohol to dissolve the adhesive. Because the robot was only run indoors rather than in an actual field, one session's samples were manually labeled to demonstrate disease detection on the dashboard.

Figure 4.8 presents the dashboard, beginning with its overall layout (Figures 4.8a and 4.8b) before turning to individual components. Figure 4.8c shows the header bar, which

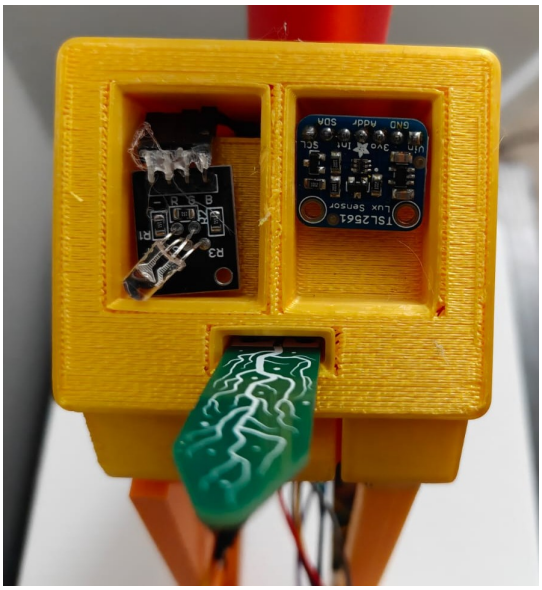


Figure 4.6: *Soil Moisture Box with all sensors and LED fitted*

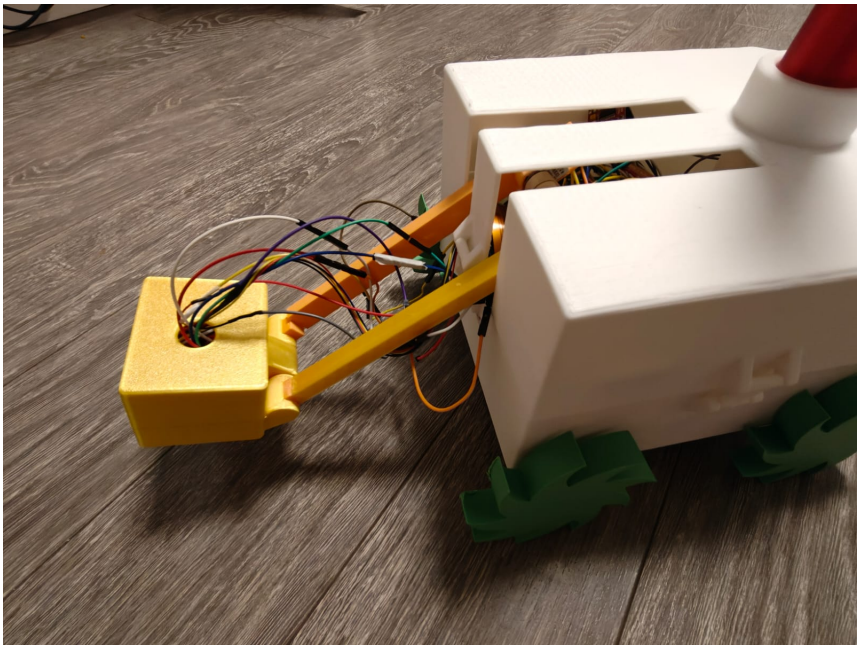
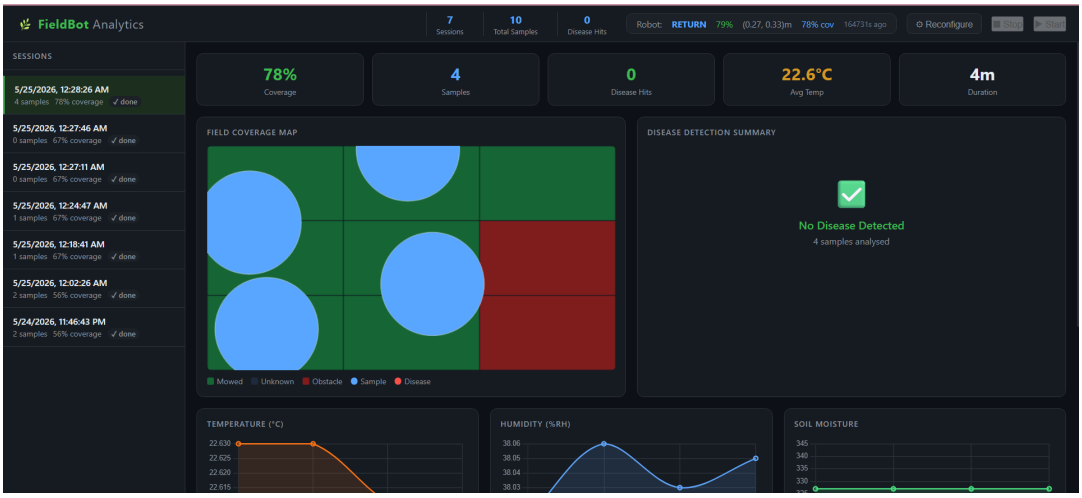


Figure 4.7: *Lowered Soil Moisture Probe*

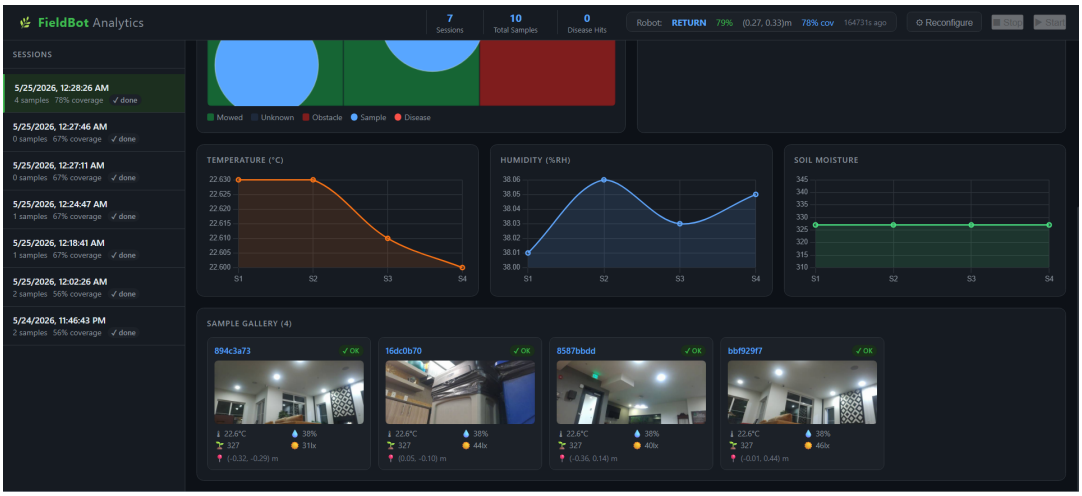
displays the robot's last known position and state, its battery percentage, the percentage of the field covered, and the time of the most recent status update. It also summarizes sample information, including the total number of samples collected across all sessions, the total number of sessions, and the number of diseased plants detected in the current session. The header bar additionally provides remote Stop and Start controls; only one of these is available at any given time depending on whether the robot is running, allowing the user to halt or resume operation remotely. Figure 4.8d shows the Disease Summary Panel for a session in which diseases were detected, while Figure 4.8e shows the corresponding image alongside the confidence score for each detected disease. Finally, Figure 4.8f shows the trend lines for temperature, humidity, and soil moisture over the course of a single session.

4.5 Costs

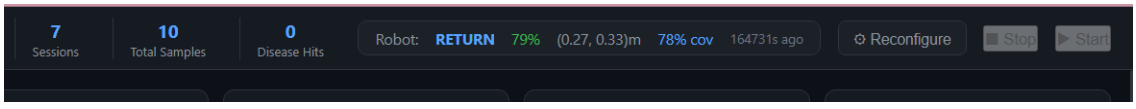
Table 4.4 summarizes the cost of all components discussed in Chapter 3. Because 3D printing was done on campus at no out-of-pocket cost, a nominal $100 USD was added to the total for fair cost comparison with commercial platforms. The complete prototype cost is significantly lower than any commercially available or research-grade farm-maintenance robot reviewed in Chapter 2, making it a feasible option for small- and medium-scale farmers.



(a) Top image of the entire dashboard

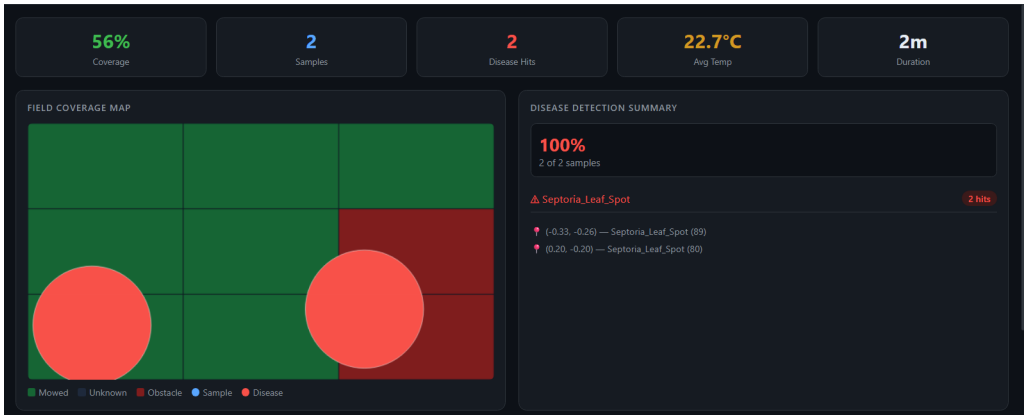


(b) Bottom image of the entire dashboard

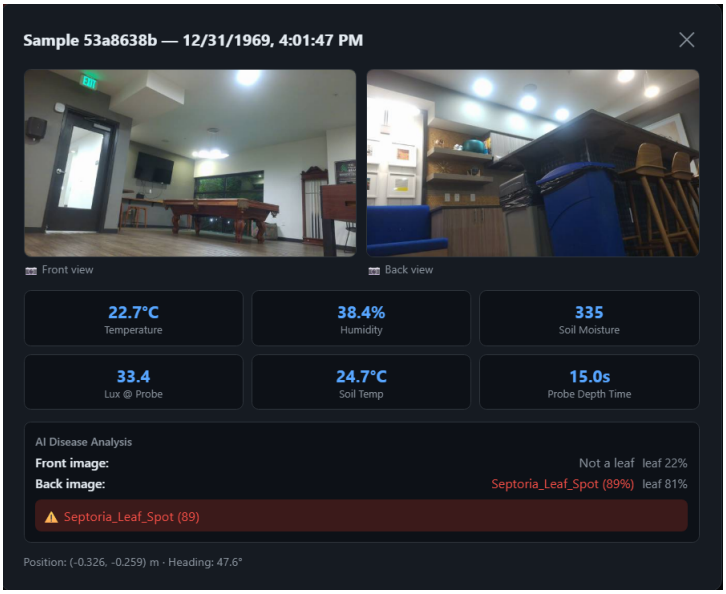


(c) Header bar

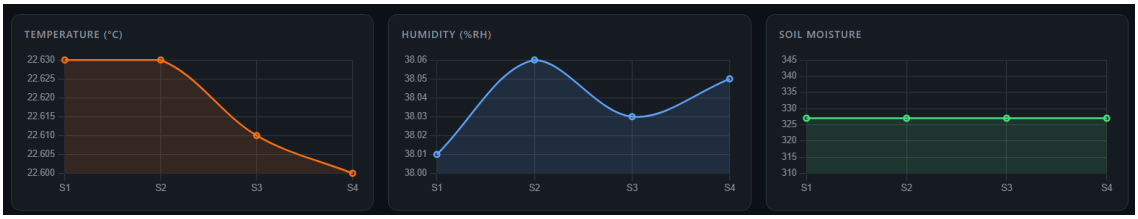
Figure 4.8: *Images of different parts of the dashboard*



(d) Session having diseases samples - disease detection summary



(e) Session having diseased samples - Sample gallery



(f) Trend lines for temperature, humidity and soil moisture

Figure 4.8: *Images of different parts of the dashboard (continued)*

Product	Quantity	Cost Per Item	Cost
Raspberry Pi 5 Active Cooler	1	13.5	13.5
Raspberry Pi AI Kit with M.2 HAT+ and Hailo AI Module	1	109.9	109.9
STEMMA QT 4-Pin Cable - 200mm	3	1.25	3.75
Si7021 Temperature & Humidity Sensor	1	9.95	9.95
IR distance sensor - GP2Y0A21YK0F	2	14.95	29.9
Raspberry Pi Camera Module 3	1	29.25	29.25
Photo Transistor Light Sensor	2	0.95	1.9
STEMMA Soil Sensor - I2C Capacitive Moisture Sensor	1	7.5	7.5
Inductive Charging Set	1	9.95	9.95
Raspberry Pi 5 FPC Camera Cable - 500mm long PID: 5820	2	2.5	5
LSM6DSOX 6 DoF Accelerometer and Gyroscope	1	11.95	11.95
STEMMA QT 4-pin Cable - 100mm	3	0.95	2.85
Lock-style Solenoid 6VDC	2	7.5	15
ALS-PT19 Analog Light Sensor	1	2.5	2.5
16-Channel 12-bit PWM/Servo Driver PCA9685	1	14.95	14.95
USB to Multi-Protocol Serial Adapter	2	21.95	43.9
DRV8871 DC Motor Driver	1	7.5	7.5
Raspberry Pi 5 18650 Battery UPS HAT (5.1V 5A)	1	60	60
N20 Motor With Encoder and Cable (Pair)	3	19.95	59.85
Amazon Basics Micro SD Memory Card	1	21.96	21.96
20 Sets JST-XH 2.54mm Pitch 2 Pin Male Female Socket	1	7.71	7.71
Power Adapter for Raspberry Pi 5	1	12.29	12.29
UGREEN SD Card Reader USB C, USB 3.0 Micro SD Card	1	6.61	6.61
TB6612FNG Dual Motor Driver Controller	1	9.92	9.92
Raspberry Pi 5 8GB	1	148.33	148.33
PCA9685 16 Channel 12-Bit PWM	1	13.99	27.98
3D Printing	1	100	100
RGB LED Module	1	11.02	11.02
5V Relay Module	1	7.71	7.71
Jumper Wires	2	11	22
		Total	800.64

Table 4.4: *Cost of different components of robot prototype (in USD)*

Chapter 5

CONCLUSION AND FUTURE WORK

This work presents the design, simulation, and implementation of a small, inexpensive agricultural robot aimed at small and medium farmers. The motivation stems from real farmer pain points, agricultural-worker health challenges, the high cost and scale of existing agricultural robots, and the need for localized field data. The proposed robot prototype combines a ∼25 cm × 25 cm ground platform, row-based Boustrophedon coverage, low-cost environmental and plant sensors, a camera-based disease detection pipeline and a cloud/frontend data workflow. The design specifically favors ground mobility because soil moisture, under-canopy inspection, physical probing, and docking-based data upload are difficult to achieve with drones alone.

The prototype was evaluated in simulation, on the AI training pipeline, and in physical field-like tests. In simulation, the boustrophedon coverage pattern was successfully implemented on a flat field, and design parameters such as the 80° IR sensor spread and approximately 25 cm camera mast height were refined through iterative trials. Spiral STC with particle swarm optimization was also evaluated but rejected in favor of boustrophedon, which was simpler to implement and better matched typical row-crop field geometry.

On the AI side, the WOA-APSO feature selector reduced 143,019 candidate features to 458, a 99.68% reduction, while still preserving discriminative information for downstream classification. Among the five classifiers evaluated, LiteCShuffle was selected for deployment based on its 97.63% accuracy, 97.39% F1 macro score, sub-2 MB footprint, and 8.9 second prediction time. Notably, the three bio-inspired classifiers (YangViT, KCNet, HybridCNN) underperformed compared to conventional CNNs, suggesting that biologically-inspired architectures designed for non-agricultural perceptual tasks do not transfer directly to leaf disease classification. Channel pruning was evaluated as a model compression strategy but reduced accuracy across all three pruned models (-2.37% to -13.61%), indicating it is not

beneficial for the lightweight architectures already considered.

In real-world tests, the assembled robot traversed a 1m × 1m flat field in approximately 3 minutes and 50 seconds, completing periodic sampling at 30-second intervals. The complete prototype, including 3D-printed body components, priced at approximately $100, was assembled for under $1000, roughly an order of magnitude less than the cheapest comparable platform reviewed in Chapter 2. The dashboard successfully displayed coverage maps, environmental sensor trends, and disease detection results from the cloud, validating the end-to-end data pipeline from field sampling to user-facing visualization.

Returning to the evaluation metrics in Table 1.1, the robot demonstrated autonomous navigation without manual intervention, sensor accuracy was confirmed through the LED – photoresistor probe insertion feedback, battery life supported a full session of approximately 40 minutes (extrapolated) of operation before docking, and the cloud upload mechanism was reliable when network connectivity was available at the dock. The modular software design (separating hardware, navigation, sampling, and data layers) supported straightforward debugging and component-level testing.

Several directions remain for future work:

- Physical field testing under varied weather, soil, and lighting conditions to validate performance beyond controlled environments.

- Improved localization through Ultra-Wideband (UWB) anchors or visual SLAM, reducing drift from odometry-only tracking.

- Multi-robot deployment to cover larger fields, reduce per-robot battery requirements, and align more closely with the decentralized swarm behavior that originally motivated the bio-inspired framing.

- More reliable docking, including robustness to misalignment and visual cues for final approach.

- A larger and more diverse disease-image dataset, especially crops outside the PlantVillage subset used here, to better evaluate generalization.

- Further exploration of bio-inspired lightweight models, particularly those designed or fine-tuned specifically for leaf-disease classification rather than adapted from unrelated tasks.

BIBLIOGRAPHY

[1] D. Khode, A. Hepat, A. Mudey, and A. Joshi, "Health-Related Challenges and Programs Among Agriculture Workers: A Narrative Review," *Cureus*, vol. 16, no. 3, e57222, ISSN: 2168-8184. DOI: 10.7759/cureus.57222. Accessed: Jul. 22, 2025. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11056488/.

[2] G. C. H. E. de Croon, J. J. G. Dupeyroux, S. B. Fuller, and J. A. R. Marshall, "Insect-inspired AI for autonomous robots," *Science Robotics*, vol. 7, no. 67, eabl6334, Jun. 2022. DOI: 10.1126/scirobotics.abl6334. Accessed: Jul. 6, 2025. [Online]. Available: https://www.science.org/doi/10.1126/scirobotics.abl6334.

[3] J. Linford and M. Haghshenas-Jaryani, "A ground robotic system for crops and soil monitoring and data collection in New Mexico chile pepper farms," en, *Discover Agriculture*, vol. 2, no. 1, p. 101, Nov. 2024, ISSN: 2731-9598. DOI: 10.1007/s44279-024-00113-3. Accessed: Jul. 23, 2025. [Online]. Available: https://doi.org/10.1007/s44279-024-00113-3.

[4] A. D. Andrushia and A. T. Patricia, "Artificial bee colony optimization (ABC) for grape leaves disease detection," en, *Evolving Systems*, vol. 11, no. 1, pp. 105–117, Mar. 2020, ISSN: 1868-6486. DOI: 10.1007/s12530-019-09289-2. Accessed: Mar. 13, 2026. [Online]. Available: https://doi.org/10.1007/s12530-019-09289-2.

[5] G. Liu and Y. Zhao, "Automated Channel Pruning Strategy Based on Artificial Bee Colony Algorithm," in *2023 6th International Conference on Computer Network, Electronic and Automation (ICCNEA)*, ISSN: 2770-7695, Sep. 2023, pp. 162–166. DOI: 10.1109/ICCNEA60107.2023.00043. Accessed: Mar. 13, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/10289015/;jsessionid=84B579A60ECBE5009ED3E3781F904F2A.

[6] A. Atefi, Y. Ge, S. Pitla, and J. Schnable, "Robotic Technologies for High-Throughput Plant Phenotyping: Contemporary Reviews and Future Perspectives," English, *Frontiers in Plant Science*, vol. 12, Jun. 2021, ISSN: 1664-462X. DOI: 10.3389/fpls.2021.611940. Accessed: Jul. 23, 2025. [Online]. Available: https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2021.611940/full.

[7] L. Cui, F. Le, X. Xue, T. Sun, and Y. Jiao, "Design and Experiment of an Agricultural Field Management Robot and Its Navigation Control System," en, *Agronomy*, vol. 14, no. 4, p. 654, Apr. 2024, Number: 4, ISSN: 2073-4395. DOI: 10.3390/agronomy14040654. Accessed: Jul. 23, 2025. [Online]. Available: https://www.mdpi.com/2073-4395/14/4/654.

[8] S. Cubero, E. Marco-Noales, N. Aleixos, S. Barbé, and J. Blasco, "RobHortic: A Field Robot to Detect Pests and Diseases in Horticultural Crops by Proximal Sensing," en, *Agriculture*, vol. 10, no. 7, p. 276, Jul. 2020, Number: 7, ISSN: 2077-0472. DOI: 10.3390/agriculture10070276. Accessed: Jul. 23, 2025. [Online]. Available: https://www.mdpi.com/2077-0472/10/7/276.

[9] *FarmDroid FD20 — Autonomous Farming Robot for Precision Agriculture*, en_US. Accessed: Jul. 23, 2025. [Online]. Available: https://farmdroid.com/products/farmdroid-fd20/.

[10] S. Yang, H. Wang, Y. Pang, Y. Jin, and B. Linares-Barranco, "Integrating Visual Perception With Decision Making in Neuromorphic Fault-Tolerant Quadruplet-Spike Learning Framework," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 3, pp. 1502–1514, Mar. 2024, ISSN: 2168-2232. DOI: 10.1109/TSMC.2023.3327142. Accessed: Jul. 11, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/10318216/.

[11] S. Vijayan and C. L. Chowdhary, "Hybrid feature optimized CNN for rice crop disease prediction," en, *Scientific Reports*, vol. 15, no. 1, p. 7904, Mar. 2025, ISSN: 2045-2322. DOI: 10.1038/s41598-025-92646-w. Accessed: Mar. 13, 2026. [Online]. Available: https://www.nature.com/articles/s41598-025-92646-w.

[12] *KCNet: An Insect-Inspired Single-Hidden-Layer Neural Network with Randomized Binary Weights for Prediction and Classification Tasks*. Accessed: Mar. 13, 2026. [Online]. Available: https://arxiv.org/html/2108.07554.

[13] S. Duhan et al., "LiteCShuffle: A lightweight and efficient deep learning approach for plant disease classification," *Cogent Food & Agriculture*, vol. 11, no. 1, p. 2 568 197, Dec. 2025, _eprint: https://doi.org/10.1080/23311932.2025.2568197, ISSN: null. DOI: 10.1080/23311932.2025.2568197. Accessed: May 27, 2026. [Online]. Available: https://doi.org/10.1080/23311932.2025.2568197.

[14] A. Bansal, S. A. Jain, and B. Khandelwal, "Bag of Tricks for Faster & Stable Image Classification," en,

[15] K. P. Jayalakshmi, V. G. Nair, and D. Sathish, "A Comprehensive Survey on Coverage Path Planning for Mobile Robots in Dynamic Environments," *IEEE Access*, vol. 13, pp. 60 158–60 185, 2025, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2025.3556446. Accessed: Jan. 26, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/10946186/.

[16] T. L. Nguyen et al., "Autonomous control for miniaturized mobile robots in unknown pipe networks," English, *Frontiers in Robotics and AI*, vol. 9, Nov. 2022, ISSN: 2296-9144. DOI: 10.3389/frobt.2022.997415. Accessed: Jul. 6, 2025. [Online]. Available: https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2022.997415/full.

[17] C. Zheng and K. Lee, "WheeLeR: Wheel-Leg Reconfigurable Mechanism with Passive Gears for Mobile Robot Applications," in *2019 International Conference on Robotics and Automation (ICRA)*, ISSN: 2577-087X, May 2019, pp. 9292–9298. DOI: 10.1109/ICRA.2019.8793686. Accessed: May 4, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8793686.

[18] *Cyberbotics: Robotics simulation with Webots*. Accessed: May 6, 2026. [Online]. Available: https://cyberbotics.com/.

[19] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," English, *Frontiers in Plant Science*, vol. 7, pp. 1419–1429, Sep. 2016, Num Pages: 11. DOI: 10.3389/fpls.2016.01419. Accessed: May 7, 2026. [Online]. Available: https://www.proquest.com/docview/3274995004/abstract/99D30E6D799B4CBDPQ/1.

[20] S. P. Bijoy, *VGG16 vs VGG19: A Detailed Comparison of the Popular CNN Architectures*, en, Oct. 2025. Accessed: May 8, 2026. [Online]. Available: https://medium.com/@sandhrabijoy/vgg16-vs-vgg19-a-detailed-comparison-of-the-popular-cnn-architectures-cae5ba404352.

[21] *Rover Pro – Rover Robotics*, Oct. 2021. Accessed: May 11, 2026. [Online]. Available: https://web.archive.org/web/20211020222456/https://roverrobotics.com/products/rover-pro.

[22] *UFACTORY xArm* -, en-US, Feb. 2023. Accessed: May 11, 2026. [Online]. Available: https://www.ufactory.cc/xarm-collaborative-robot.

[23] *Amazon.com: Intel RealSense Depth Camera D435i, Silver, 1080p Video Capture Resolution (82635D435IDK5P) : Electronics*. Accessed: May 11, 2026. [Online]. Available: https://www.amazon.com/Intel-RealSense-Depth-Camera-D435i/dp/B07MWR2YJB.

[24] *RPLiDAR S1 Portable ToF Laser Scanner Kit - 40M Range*, en. Accessed: May 11, 2026. [Online]. Available: https://www.seeedstudio.com/RPLiDAR-S1-360-Degree-Laser-Scanner-Kit-40M-Range-p-4008.html.

[25] *simpleRTK2B Budget*, en-US. Accessed: May 11, 2026. [Online]. Available: https://www.ardusimple.com/product/simplertk2b/.

[26] *WITMOTION WT901BLECL MPU9250 High-Precision 9-Axis Gyroscope, Accelerometer, Magnetometer, 3-Axis Bluetooth AHRS IMU Sensor for Arduino*, en. Accessed: May 11, 2026. [Online]. Available: https://www.robotshop.com/products/witmotion-witmotion-wt901blecl-mpu9250-high-precision-9-axis-gyroscope-accelerometer-magnetometer-3-axis-bluetooth-ahrs-imu-sensor-arduino.

[27] *TEROS 12 - METER Group*, en-US. Accessed: May 11, 2026. [Online]. Available: https://metergroup.com/products/teros-12/.

[28] *NEW IN BOX Intel BOXNUC6CAYH NUC Kit Components Black 735858320344*, en-us. Accessed: May 11, 2026. [Online]. Available: https://www.ebay.com/itm/377126620824.

[29] *Arduino Mega 2560 Rev3*, en. Accessed: May 11, 2026. [Online]. Available: https://store-usa.arduino.cc/products/arduino-mega-2560-rev3.

[30] *Tattu Plus 12000mAh 6s 15C 22.2V Smart Lipo Battery Pack with EC5 Plug (New Version)*, en. Accessed: May 11, 2026. [Online]. Available: https://genstattu.com/tattu-plus2-15c-12000mah-6s1p-ec5-smart-lipo-battery.html.

[31] *Clearpath Husky A200*, en-GB. Accessed: May 11, 2026. [Online]. Available: https://hotrobotics.co.uk/equipment/clearpath-husky-a200/.

[32] R. Inc, *AgileX Scout Mini Rover — RobotLAB*, en. Accessed: May 11, 2026. [Online]. Available: https://www.robotlab.com/higher-ed-robots/store/scout-mini.

Appendix A

ABBREVIATIONS

Abbreviation	**Full Form**
4WD	4 Wheel Drive
ASM	Angular Second Moment
CPP	Coverage Path Planning
DOF	Degrees of Freedom
FAA	Federal Aviation Administration
FLOPS	Floating Point Operations
FSM	Finite State Machine
GLCM	Gray-Level Co-occurrence Matrix
GNSS	Global Navigation Satellite System
HSI	Hue, Saturation, Intensity
HSV	Hue, Saturation, Value
IDM	Inverse Difference Moment
IMU	Inertial Measurement Unit
LAB	CIELAB or Commission Internationale de l'Éclairage
LFNT	Link-Fault Neuromorphic Tolerant
LiDAR	Light Detection and Ranging
LUV	CIELUV
MDPI	Multidisciplinary Digital Publishing Institute
NDVI	Normalized Difference Vegetation Index
ROS2	Robot Operating System 2
RTK	Real-Time Kinematic
SLAM	Simultaneous Localization and Mapping
UGV	Unmanned Ground Vehicle
Vis/NIR	Visible and Near-Infrared

Appendix B

ROBOT COST ESTIMATION

For robots where no cost was reported, the cost is derived from publicly listed component prices at the time of writing (May 2025), or benchmarked against comparable commercial platforms reported in the literature. Labor, software development, and institutional overhead are excluded from all estimates.

B.1 NMSU Mobile Manipulator[3]

The major components of this system are explicitly identified in Linford and Haghshenas-Jaryani[3] and are commercially available. The estimate of $12,900–$18,000 USD is derived from the publicly listed prices of these components:

- 4WD Rover Pro mobile base (Rover Robotics, Wayzata, MN, USA)[21] — $5,000–$7,000
- xArm6 6-DoF robotic manipulator (Ufactory, Shenzhen, China)[22] — $6,000–$8,000
- Intel RealSense D435i RGB+Depth camera ×2[23] — $350–$450 each
- RPLIDAR S1 LiDAR unit[24] — $600–$700
- simpleRTK2B RTK GNSS module (Ardusimple, Andorra)[25] — $200–$250
- WT901BLECL 9-axis IMU (WitMotion, Shenzhen, China)[26] — $40–$50
- TEROS 12 soil moisture and EC sensor (METER Group, Pullman, WA, USA)[27] — $250–$350
- Intel NUC mini PC[28] — $150–$200
- Arduino Mega 2560 and associated data-logging electronics[29] — $40–$60
- TATTU Plus 12,000 mAh LiPo smart battery pack[30] — $250–$350

- 3D-printed mounts and miscellaneous hardware — $300–$500

B.2 Multifunctional Field Management Robot[7]

Cui et al. describe a custom-fabricated chassis with an adjustable wheel-track mechanism, hub-motor drive units, and an on-board computing and vision system; no component-level prices are provided. Because the drivetrain and chassis were purpose-built for the study, direct component pricing is not available. The cost estimate of $8,000–$15,000 USD is therefore benchmarked against comparable commercially available agricultural UGV platforms of similar size, payload capacity, and sensor configuration reported in the literature:

- The Clearpath Robotics Husky A200, a widely cited medium-duty research UGV with comparable payload (∼75 kg) and four-wheel differential drive, is listed at approximately $19,900 USD[31], providing an upper bound for a fully integrated off-the-shelf system.
- The AgileX Scout Mini, a smaller four-wheel-drive research platform, is listed at approximately $6,900 USD[32], providing a lower bound for the base mobility platform alone.

Accounting for a custom chassis and wheel-track mechanism ($3,000–$5,000), four hub-motor drive units and controllers ($2,000–$4,000), on-board computing and vision ($1,000–$2,000), GNSS ($200–$500), battery system ($500–$1,000), and structural and miscellaneous electronics ($1,000–$2,000), the estimate of $8,000–$15,000 is consistent with these benchmarks. This estimate applies to a single research prototype and would differ substantially in commercial production.